

Міністерство освіти і науки України
Національний університет “Львівська політехніка”

Кафедра ЕОМ



Курсовий проект

З дисципліни «Програмні засоби мікропроцесорних систем»

на тему:

"Програмне забезпечення мікрокомп'ютера для збору телеметрії SMBus/PMBus пристроїв"

Виконав:

ст. гр. КІ-405

Симанишин В. П.

Перевірив:

Пуйда Д. В.

Львів-2026

АНОТАЦІЯ

У цій курсовій роботі розроблено програмне забезпечення мікрокомп'ютера для опитування PMBus-пристроїв через I2C/SMBus, кодування телеметричних даних у JSON та передачі їх на MQTT broker через Wi-Fi. Реалізовано багатозадачну архітектуру на FreeRTOS, механізм черг IPC, облік метрик продуктивності, подієву діагностику та механізм store-and-forward із RAM буфером і опційним flash-рівнем для роботи в офлайн-режимі.

Практична цінність результату полягає у можливості використання розробленого gateway як базової платформи для побудови систем віддаленого моніторингу джерел живлення і суміжних вбудованих систем з вимогами до телеметрії в реальному часі.

ЗМІСТ

АНОТАЦІЯ	2
ЗМІСТ	3
ПЕРЕЛІК СКОРОЧЕНЬ	6
ТЕХНІЧНЕ ЗАВДАННЯ	7
1. Найменування роботи.....	7
2. Підстава для виконання.....	7
3. Мета роботи	7
4. Вихідні дані.....	7
5. Функціональні вимоги	7
6. Вимоги до архітектури ПЗ	8
7. Вимоги до тестування.....	8
8. Цільові числові показники	8
9. Очікувані результати	8
ВСТУП	10
РОЗДІЛ 1. РОЗРОБЛЕННЯ СХЕМИ ЕЛЕКТРИЧНОЇ ФУНКЦІОНАЛЬНОЇ	12
1.1. Аналіз відомих прототипів та аналогів.....	12
1.2. Структура мікрокомп'ютера та обґрунтування вибору апаратної платформи шлюзу	15
1.3. Цільова підсистема керування живленням	16
1.4. Організація інтерфейсів та протоколів обміну інформацією	16
1.5. Опис схеми електричної функціональної та основних режимів роботи	17

	4
1.6. Висновки до розділу 1	18
РОЗДІЛ 2. РОЗРОБЛЕННЯ АРХІТЕКТУРИ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	19
2.1. Загальна концепція та модульна структура ПЗ.....	19
2.2. Багатозадачна модель та міжпроцесна взаємодія (IPC).....	19
2.3. Модель даних та MQTT-контракти.....	21
2.4. Підсистема буферизації (Store-and-Forward).....	22
2.5. UML-моделювання архітектури ПЗ	24
2.6. Висновки до розділу 2	26
РОЗДІЛ 3. РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ МІКРОКОМП'ЮТЕРА МПС.....	27
3.1. Середовище розробки та інструментальні засоби.....	27
3.2. Модуль ініціалізації мікрокомп'ютера	29
3.3. Алгоритм роботи драйвера та задачі опитування PMBus	34
3.4. Алгоритм мережевої взаємодії та формування JSON-контрактів	38
3.5. Програмна реалізація підсистеми буферизації	42
3.6. Висновки до розділу 3	46
РОЗДІЛ 4. РОЗРОБЛЕННЯ МЕТОДИКИ ТЕСТУВАННЯ ПЗ	47
4.1. Мета, об'єкт і завдання тестування ПЗ	47
4.2. Тестове оточення та інструментальні засоби.....	48
4.3. Методика модульного тестування host-side	50
4.4. Методика інтеграційного та runtime-тестування	52
4.5. Результати тестування, критерії pass/fail та інтерпретація.....	57
4.6. Висновки до розділу 4	60

ВИСНОВКИ.....	62
СПИСОК ЛІТЕРАТУРИ.....	64
ДОДАТКИ.....	66

ПЕРЕЛІК СКОРОЧЕНЬ

- API - application programming interface
- BSP - board support package
- CRC - cyclic redundancy check
- FIFO - first in, first out
- Flash - флеш-пам'ять
- Gateway - шлюз
- FreeRTOS - free and open-source real-time operating system kernel
- HAL - hardware abstraction layer
- I2C - inter-integrated circuit
- IPC - inter-process communication
- JSON - JavaScript Object Notation
- MQTT - message queuing telemetry transport
- PEC - packet error checking
- PDL - peripheral driver library
- PMBus - power management bus
- PSoC - programmable system-on-chip
- QoS - quality of service
- RAM - random access memory
- RTOS - real-time operating system
- SCB - serial communication block
- SMBus - system management bus
- UART - universal asynchronous receiver-transmitter
- МПІС - мікропроцесорна система

ТЕХНІЧНЕ ЗАВДАННЯ

1. Найменування роботи

Програмне забезпечення мікрокомп'ютера для збору телеметрії SMBus/PMBus пристроїв

2. Підстава для виконання

Курсова робота з дисципліни «Програмні засоби мікропроцесорних систем».

3. Мета роботи

Розробити програмне забезпечення для мікрокомп'ютера, яке забезпечує:

1. Опитування PMBus-пристрою через I2C/SMBus.
2. Декодування даних живлення та стану.
3. Передавання телеметрії на MQTT broker через Wi-Fi.
4. Стійку роботу при втраті каналу зв'язку завдяки буферизації.

4. Вихідні дані

1. Платформа gateway: CY8CKIT-062S2-43012 (PSoC6 + CYW43012).
2. Периферійний вузол: PMBus target KIT_PSC3M5_EVK, адреса 0x58.
3. Інтерфейси: I2C/SMBus, Wi-Fi, UART debug.
4. Протокол верхнього рівня: MQTT (broker у локальній мережі).
5. Базова конфігурація: rtos_test/source/profiles/profile_default.h.

5. Функціональні вимоги

1. Виконати ініціалізацію BSP, UART та RTOS-планувальника.
2. Реалізувати драйвер PMBus-команд: VOUT_MODE, STATUS_*, READ_VIN/VOUT/IIN/IOUT/TEMP/POUT.
3. Реалізувати періодичне опитування пристрою та формування записів telemetry/status.
4. Реалізувати MQTT-публікацію з topic/payload контрактами.
5. Реалізувати механізм store-and-forward: RAM буфер + опційний flash-рівень.
6. Реалізувати збір метрик продуктивності та подій.

6. Вимоги до архітектури ПЗ

1. Багатозадачність на FreeRTOS: pmbus_poll_task, mqtt_gw_task, buffer_task.
2. Обмін між задачами через gateway_ipc черги.
3. Модульний поділ за підсистемами: init, driver, decode, transport, buffering, metrics, events.

7. Вимоги до тестування

1. Host unit tests повинні завершуватись без помилок:
 - test_buffer_ring
 - test_pmbus_decode
 - test_json_encode
2. Перевірити коректність JSON-контрактів.
3. Перевірити поведінку буфера в офлайн-режимі за логами та графіками.

8. Цільові числові показники

1. Кількість одночасно опитуваних PMBus target-пристроїв у штатному профілі - не менше 2.
2. Квантиль затримки read_to_publish_p95 у штатному online-режимі для профілю profile_default.h - не більше 150 ms.
3. Сума критичних помилок обміну в контрольному online-сеансі (помилки опитування + помилки черг + помилки публікації) = 0.

9. Очікувані результати

1. Працюючий firmware gateway.
2. Пояснювальна записка за методичними вимогами.
3. Графічна частина: функціональна схема, спрощена принципова схема, UML, блок-схеми алгоритмів.
4. Пакет матеріалів для захисту в електронному форматі.

ВСТУП

Розвиток вбудованих мікропроцесорних систем для енергетики та промислової автоматизації наряду пов'язаний із задачами дистанційного моніторингу, оперативної діагностики та накопичення телеметричних даних. На рівні інтерфейсу до силових підсистем широко застосовуються PMBus/SMBus та I2C, тоді як у мережевій інтеграції домінує модель publish/subscribe на базі MQTT [2], [3], [4], [5], [6]. Для практичного впровадження таких систем важливо підтвердити вибір протоколів і архітектурних рішень не лише інженерною практикою, а й результатами досліджень продуктивності та масштабованості [12], [13].

У межах курсового проєкту розглядається розроблення ПЗ мікрокомп'ютера, що виконує роль PMBus-MQTT gateway: опитує PMBus-пристрої по I2C/SMBus, декодує значення телеметрії, формує JSON-повідомлення та передає їх на MQTT broker через Wi-Fi [2], [4], [5], [6], [7]. Реалізація опирається на RTOS-підхід із поділом на окремі задачі опитування, публікації та сервісної обробки черг, що відповідає типовим практикам побудови детермінованих вбудованих систем [8], [9].

Мета курсової роботи: розробити програмне забезпечення мікрокомп'ютера для надійного збору PMBus-телеметрії та її передачі в MQTT-середовище з підтримкою store-and-forward буферизації у випадку нестабільного мережевого з'єднання [1], [6], [12].

Для досягнення мети поставлено такі задачі:

1. Реалізувати програму ініціалізації апаратних та програмних підсистем (BSP, UART, RTOS, IPC, буферизація).
2. Реалізувати драйвер PMBus master із підтримкою retry, тайм-аутів та PEC.
3. Реалізувати періодичний алгоритм опитування PMBus-команд і формування структур телеметрії/статусу.
4. Реалізувати формування MQTT topic/payload згідно контрактів даних.
5. Реалізувати механізм буферизації та відкладеної публікації (RAM/flash).
6. Побудувати методику тестування з формальними критеріями pass/fail.

Об'єкт дослідження: процес обміну телеметричними даними між RMBus-пристроєм і MQTT-інфраструктурою в мікропроцесорній системі.

Предмет дослідження: архітектура і програмні засоби RMBus-MQTT gateway, зокрема модулі ініціалізації, драйвер RMBus, задачі опитування/публікації, механізми буферизації, а також засоби контролю метрик і подій [8], [9], [10], [11].

Практична цінність роботи полягає у підготовці відтворюваного рішення для вбудованого шлюзу, яке може бути використане як база для лабораторних та дослідницьких стендів моніторингу джерел живлення. Додатково результат придатний для масштабування: від одиночного RMBus target до конфігурацій із кількома пристроями та різними профілями навантаження.

Структурно курсова робота складається з чотирьох технічних розділів: функціональна схема системи, архітектура ПЗ, розроблення ПЗ мікрокомп'ютера, методика тестування; також містить висновки, список літератури та додатки з графічними матеріалами і лістингами [1].

РОЗДІЛ 1. РОЗРОБЛЕННЯ СХЕМИ ЕЛЕКТРИЧНОЇ ФУНКЦІОНАЛЬНОЇ

1.1. Аналіз відомих прототипів та аналогів

Перед вибором власної архітектури PMBus-MQTT gateway доцільно розглянути класи рішень, які вже використовуються для роботи з I2C/SMBus/PMBus та для мережевої доставки телеметрії. Для курсового проекту принципово важливими є такі критерії порівняння: спеціалізація під PMBus/SMBus, автономність роботи без ПК, наявність штатної MQTT-інтеграції, підтримка буферизації при втраті зв'язку, а також придатність до перенесення на мікроконтролерну платформу з RTOS [4], [6].

Узагальнений аналіз найбільш показових прототипів та аналогів наведено в табл. 1.1.

Таблиця 1.1 - Порівняння відомих прототипів і аналогів PMBus -> MQTT

Прототип / клас рішень	Характерні параметри	Сильні сторони	Обмеження відносно задачі курсового проекту
Aardvark I2C/SPI Host Adapter [16]	USB-host адаптер; I2C 1-800 kHz; Windows/Linux/macOS; сумісність із SMBus/PMBus	Зручний для bring-up, ручного читання/запису команд і швидкої локальної діагностики шини	Потребує ПК, не є автономним edge-вузлом, не має вбудованої MQTT-доставки й store-and-forward
Beagle I2C/SPI Protocol Analyzer [17]	Пасивний моніторинг I2C до 4 MHz; декодування трафіку в реальному часі; робота з ПК	Добре підходить для аналізу таймінгів, АСК/НАК, restart/stop та інших проблем шини	Це діагностичний інструмент, а не шлюз збору телеметрії; MQTT і буферизація відсутні

Прототип / клас рішень	Характерні параметри	Сильні сторони	Обмеження відносно задачі курсового проєкту
Fusion Digital Power Designer від TI [18]	GUI для вибраних digital power контролерів; зв'язок по PMBus через TI USB adapter; моніторинг VIN/VOUT/IOU/TEMP і попереджень/аварій	Дає готовий інтерфейс моніторингу цифрового живлення та швидкий старт у межах vendor ecosystem	Орієнтований на ПК і сумісні TI-пристрої; не є універсальним MCU+RTOS шлюзом із власним MQTT-контрактом
Linux hwmon/pmbus [19]	Системний драйвер PMBus; експорт сенсорів через sysfs; явна інстанціяція девайсів у Linux	Дає готовий стек читання PMBus-даних на Linux-платформах і зручну інтеграцію з userspace	Потребує Linux-платформи; не виконує безпечного auto-probe PMBus-пристроїв; MQTT потребує окремого ПЗ
HiveMQ Edge / MQTT edge gateway [20]	Протокольний конвертер у MQTT; MQTT; Offline Buffer із записом на диск; standard hardware	Сильний з боку мережевої інтеграції та доставки telemetry у MQTT-інфраструктуру	Не спеціалізований під PMBus/SMBus, орієнтований на значно «важчу» платформу, ніж MCU+FreeRTOS шлюз

Із наведеного порівняння видно, що наявні аналоги добре закривають або локальну лабораторну діагностику шини, або загальну задачу protocol-to-MQTT інтеграції на Linux/edge-платформах. Водночас вони не дають потрібного поєднання властивостей для цього проєкту: PMBus-спеціалізації, автономної роботи без ПК, детермінованої поведінки на MCU під керуванням RTOS та власних контрольованих telemetry/status/metrics контрактів.

Отже, розроблюваний шлюз займає проміжну нішу між діагностичними інструментами та «важкими» edge-платформами. Він має поєднати переваги

спеціалізованого PMBus-доступу з автономністю, мережевою MQTT-публікацією та відтворюваними експериментальними метриками, що і визначає подальший вибір апаратної платформи й архітектури ПЗ.



Рис. 1.1. Приклади відомих апаратних і програмних прототипів: Aardvark I2C/SPI Host Adapter, Beagle I2C/SPI Protocol Analyzer.

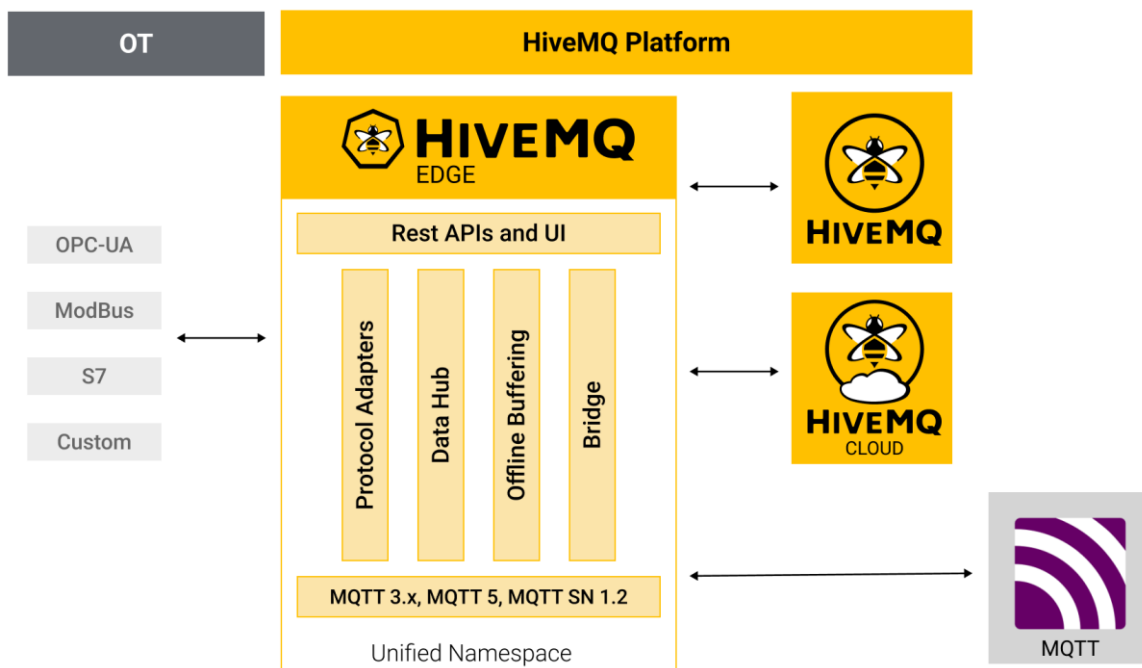


Рис. 1.2. Програмні аналоги інтеграції PMBus telemetry: HiveMQ Edge.

1.2. Структура мікрокомп'ютера та обґрунтування вибору апаратної платформи шлюзу

Для реалізації крайового шлюзу (Edge gateway), який забезпечує збір телеметрії з пристроїв живлення та її передачу до MQTT-інфраструктури, необхідна апаратна платформа з такими характеристиками:

1. достатня продуктивність для багатозадачного виконання під керуванням RTOS;
2. наявність апаратних інтерфейсів I2C/SMBus і UART;
3. підтримка мережевої взаємодії через Wi-Fi та TCP/IP стек.

У межах проєкту як базову платформу обрано відлагоджувальну плату CY8CKIT-062S2-43012 [15], яка виконує роль мікрокомп'ютера шлюзу.

Вибір цієї платформи обґрунтовується такими факторами:

1. Продуктивність: ресурсів мікроконтролерної платформи достатньо для детермінованого виконання задач `pmbus_poll_task`, `mqtt_gw_task` і `buffer_task` під керуванням FreeRTOS [8], [9].
2. Комунікаційні можливості: апаратна підтримка I2C забезпечує можливість опитування PMBus-пристроїв, а Wi-Fi підсистема дає змогу організувати доступ до MQTT-брокера [6], [11], [15].
3. Пам'ять: доступний обсяг RAM/Flash дозволяє реалізувати механізм store-and-forward буферизації телеметрії при втраті мережевого з'єднання.

Підсистеми, що входять до складу мікрокомп'ютера шлюзу:

1. обчислювальне ядро та периферія мікроконтролера;
2. I2C інтерфейс для взаємодії з PMBus target;
3. Wi-Fi мережевий інтерфейс для MQTT обміну;
4. UART інтерфейс для налагоджувальної телеметрії.

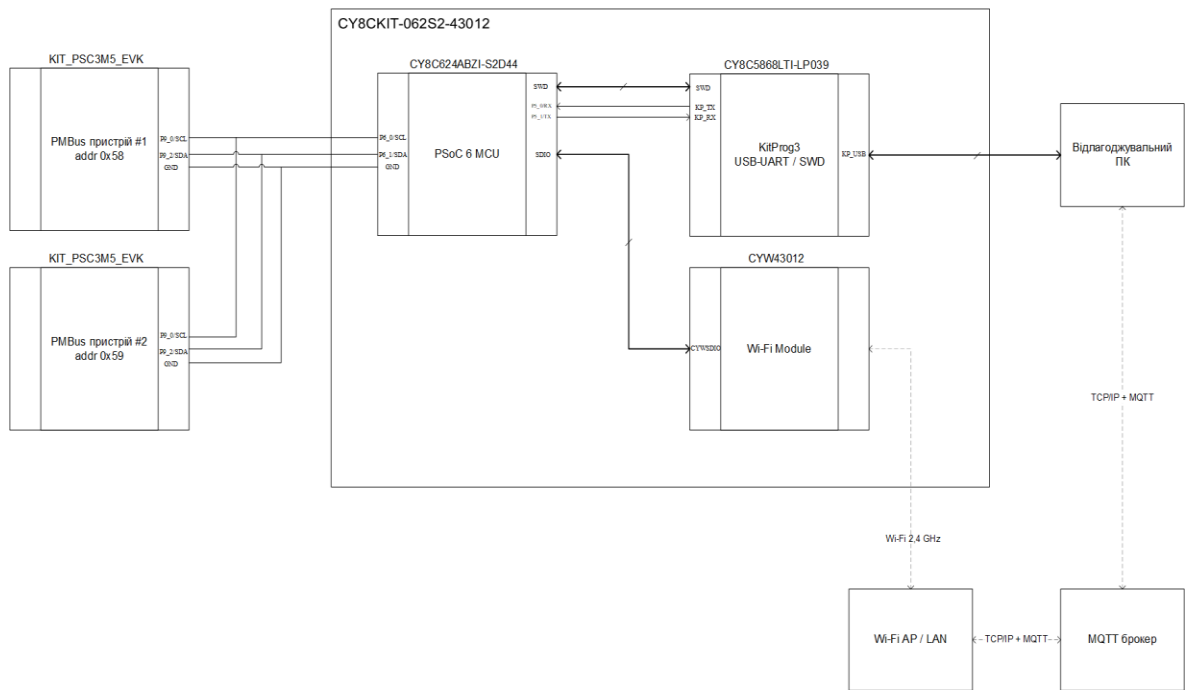


Рис. 1.3. Функціональна електрична схема мікрокомп'ютера шлюзу та його зовнішніх зв'язків.

1.3. Цільова підсистема керування живленням

Джерелом первинних даних для шлюзу є периферійний цільовий пристрій KIT_PSC3M5_EVK, який у межах стенду працює як PMBus/SMBus slave-пристрій з адресою 0x58.

Цільова підсистема апаратно підключена до шлюзу через лінії SCL, SDA, GND інтерфейсу I2C/SMBus. На логічному рівні шлюз циклічно ініціює читання команд телеметрії та статусу.

Функціонально підсистема керування живленням забезпечує:

1. зняття телеметричних параметрів: READ_VIN, READ_VOUT, READ_IIN, READ_IOUT, READ_TEMPERATURE_1, READ_POUT;
2. зчитування статусних регістрів групи STATUS_* для виявлення аварійних або попереджувальних станів.

Така декомпозиція на два вузли (gateway і target) дозволяє ізолювати контур мережевої взаємодії від контуру вимірювання/моделювання параметрів живлення, що підвищує керованість системи та її відмовостійкість [2], [4], [5].

1.4. Організація інтерфейсів та протоколів обміну інформацією

У розробленій системі реалізовано декілька рівнів обміну даними:

1. Локальна шина I2C/SMBus: фізичний канал між gateway і target; підтримує підключення кількох пристроїв у межах спільної шини [4], [5].
2. Протокол PMBus: прикладний рівень поверх SMBus; шлюз працює як master і ініціює транзакції читання, а перевірка цілісності даних виконується через PEC (Packet Error Checking) [2], [4].
3. Мережевий інтерфейс Wi-Fi: канал підключення шлюзу до локальної мережі з виходом на MQTT broker.
4. Протокол MQTT: передача даних у моделі publish/subscribe; шлюз публікує JSON-повідомлення у топіки телеметрії, статусу, подій і метрик [6], [7], [12], [13].
5. UART інтерфейс: службовий канал для виведення логів, діагностики та супроводу експериментів.

1.5. Опис схеми електричної функціональної та основних режимів роботи

Схема електрична функціональна (див. Додаток А) відображає центральний вузол шлюзу CY8CKIT-062S2-43012, підключення PMBus target по лініях SDA/SCL, а також мережеву взаємодію з MQTT broker.

З урахуванням вимог до надійності моніторингу реалізовано такі режими роботи:

1. Режим ініціалізації: запуск BSP і периферії, ініціалізація RTOS-підсистем, конфігурація I2C/SMBus, встановлення мережевого підключення та MQTT сесії [8], [9].
2. Нормальний режим (Online): періодичне опитування PMBus target, формування JSON та публікація повідомлень на MQTT broker у реальному часі [6], [7].
3. Автономний режим буферизації (Offline): при втраті мережі опитування PMBus не зупиняється, а сформовані записи накопичуються у RAM/Flash буфері за принципом FIFO.

4. Режим відновлення (Recovery): після відновлення з'єднання накопичені записи вивантажуються на broker батчами, після чого система повертається до штатного online-циклу.

Описані режими відповідають практиці побудови відмовостійких edge-рішень, де локальний збір даних має бути безперервним навіть при нестабільному каналі передачі [12], [13].

1.6. Висновки до розділу 1

У Розділі 1 сформовано функціональну основу апаратної частини системи PMBus-MQTT gateway та обґрунтовано вибір платформи реалізації. Показано, що використання CY8CKIT-062S2-43012 як центрального вузла шлюзу забезпечує необхідні обчислювальні та комунікаційні ресурси для виконання задач телеметрії в реальному часі, мережевої публікації та буферизації даних.

Визначено склад цільової підсистеми керування живленням і її роль як джерела PMBus-даних, а також зафіксовано інтерфейсну модель взаємодії між вузлами: I2C/SMBus на локальному рівні та MQTT over Wi-Fi на мережевому рівні. Структура обміну відповідає вимогам стандартів PMBus/SMBus/I2C і MQTT [2], [3], [4], [5], [6].

Описані режими функціонування (ініціалізація, online, offline, recovery) підтверджують придатність вибраної структури до побудови відмовостійкої системи моніторингу, у якій втрата каналу зв'язку не призводить до втрати процесу збору даних.

Отримані результати Розділу 1 є достатніми для переходу до Розділу 2, де деталізується програмна архітектура шлюзу та взаємодія його модулів.

РОЗДІЛ 2. РОЗРОБЛЕННЯ АРХІТЕКТУРИ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1. Загальна концепція та модульна структура ПЗ

Архітектура програмного забезпечення gateway побудована на базі FreeRTOS, що дозволяє розділити незалежні функції системи на окремі задачі з фіксованими пріоритетами та передбачуваною часовою поведінкою [8], [9]. Такий підхід є критичним для системи, у якій одночасно виконуються:

1. циклічне опитування PMBus-пристроїв;
2. мережеве підключення та MQTT-публікація;
3. буферизація даних при втраті каналу зв'язку.

Логічно програмне забезпечення розділено на такі рівні:

1. Рівень апаратних абстракцій (BSP/HAL/PDL): ініціалізація платформи, периферії, I2C-блоку, мережевого стека [10], [11].
2. Рівень драйверів: PMBus master (pmbus_master.*) з підтримкою retry/timeout/PEC [2], [4], [5].
3. Рівень RTOS-задач: pmbus_poll_task, mqtt_gw_task, buffer_task.
4. Рівень прикладних сервісів: декодування PMBus, формування JSON, облік метрик, події, буферизація, контракти MQTT.

Окремо виділено дві прошивки:

1. rtos_test (gateway) - збір, обробка і передача телеметрії;
2. target_proj (target) - PMBus slave-модель джерела живлення.

Така ізоляція підвищує керованість розробки: мережеву логіку можна змінювати незалежно від моделі target, а PMBus-поведінку тестувати окремо від MQTT-транспорту.

2.2. Багатозадачна модель та міжпроцесна взаємодія (IPC)

У runtime-системі gateway використовуються три основні задачі:

1. pmbus_poll_task (пріоритет 4): ініціалізує PMBus master, виконує періодичне опитування команд телеметрії та статусу, формує записи і передає їх у IPC-черги.

2. `mqtt_gw_task` (пріоритет 3): керує Wi-Fi/MQTT з'єднанням, реалізує reconnect з exponential backoff, читає IPC-черги, публікує JSON-повідомлення, а також запускає flush буферизованих записів.
3. `buffer_task` (пріоритет 2): виконує фоновий супровід буфера (оновлення gauge-метрик глибини RAM/Flash); безпосередню MQTT-публікацію не виконує.

Основний IPC реалізовано в модулі `gateway_ipc` через три черги:

1. `telemetry_queue` - для telemetry-records;
2. `status_queue` - для status-records;
3. `event_queue` - для event-records.

Додатково в `gateway_ipc` підтримуються:

1. глобальний seq-лічильник;
2. прапор стану `mqtt_online`;
3. функція формування часової мітки `now_ms`.

Ця модель забезпечує слабке зв'язування між задачами: producer (poll task) не залежить від мережевого стану, а transport-логіка (MQTT task) працює зі стандартизованими структурами даних.

Параметри задач у поточній реалізації:

1. `pmbus_poll_task`: stack 1024, priority 4;
2. `mqtt_gw_task`: stack 3072, priority 3;
3. `buffer_task`: stack 1024, priority 2.

Фрагмент коду 2.1 - створення задач у `main.c`:

```
/* Task A - PMBus polling */
xTaskCreate(pmbus_poll_task, "PMBus_Poll",
            PMBUS_POLL_TASK_STACK_SIZE, NULL,
            PMBUS_POLL_TASK_PRIORITY, NULL);

/* Task B - MQTT gateway */
xTaskCreate(mqtt_gw_task, "MQTT_GW",
            MQTT_GW_TASK_STACK_SIZE, NULL,
            MQTT_GW_TASK_PRIORITY, NULL);

/* Task C - Buffer task */
```

```
xTaskCreate(buffer_task, "Buffer",
          BUFFER_TASK_STACK_SIZE, NULL,
          BUFFER_TASK_PRIORITY, NULL);
```

Фрагмент коду 2.2 - ініціалізація IPC-черг у gateway_ipc.c:

```
s_telemetry_q = xQueueCreate(IPC_TELEMETRY_QUEUE_DEPTH,
                             sizeof(telemetry_record_t));
s_status_q    = xQueueCreate(IPC_STATUS_QUEUE_DEPTH,
                             sizeof(status_record_t));
s_event_q     = xQueueCreate(IPC_EVENT_QUEUE_DEPTH,
                             sizeof(event_record_t));

if (s_telemetry_q == NULL || s_status_q == NULL || s_event_q == NULL)
{
    printf("[IPC] ERROR: Failed to create queues\n");
    return false;
}
```

З урахуванням фіксованих глибин черг (64/16/16) ця схема забезпечує передбачуване використання RAM і контрольовану деградацію при піковому навантаженні.

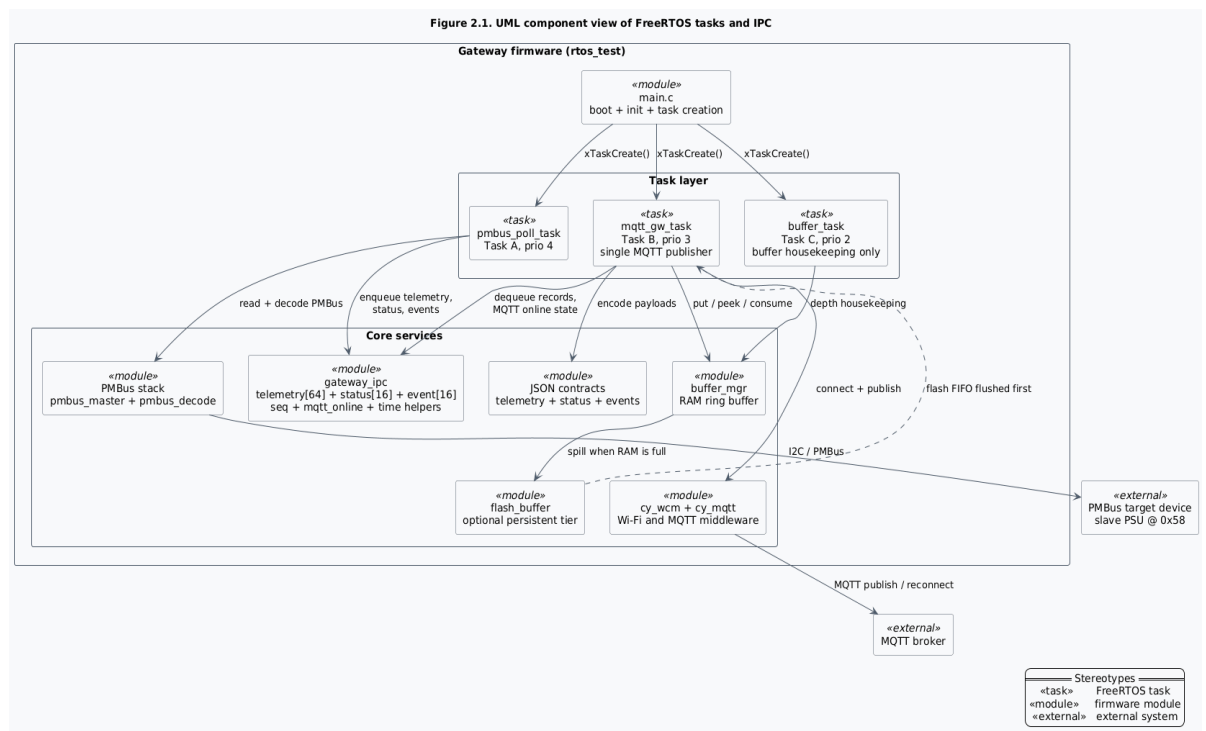


Рис. 2.1. Схема взаємодії задач FreeRTOS та IPC-черг у gateway.

2.3. Модель даних та MQTT-контракти

Потік даних у gateway організовано як послідовне перетворення:

1. PMBus-транзакції повертають сирі байти регістрів;
2. модуль декодування переводить значення у інженерні одиниці;
3. формується типізована структура (telemetry/status/event);
4. виконується JSON-кодування та MQTT-публікація.

Для передачі даних застосовано модель single-publisher: усі publish операції серіалізовані в mqtt_gw_task. Це спрощує контроль QoS, повторних спроб і відновлення після outage [6], [7].

Фрагмент коду 2.3 - шлях telemetry-record у mqtt_gw_task.c:

```
while (xQueueReceive(gateway_ipc_telemetry_queue(), &rec, 0) == pdTRUE)
{
    int json_len = encode_telemetry_json(&rec, s_json_buf, JSON_BUF_SIZE);
    if (json_len <= 0) { continue; }

    int topic_len = build_device_topic(s_topic_buf, TOPIC_BUF_SIZE,
                                      rec.addr_7bit, "telemetry");
    if (topic_len <= 0) { continue; }

    if (!publish_json(s_topic_buf, s_json_buf, (size_t)json_len))
    {
        buffer_mgr_put(s_topic_buf, s_json_buf, (uint16_t)json_len);
    }
}
```

Наведений фрагмент демонструє ключовий архітектурний інваріант: дані не втрачаються при невдалій публікації, а переводяться в підсистему буферизації.

Логічно потоки розділяються на:

1. телеметрію (напруга, струм, температура, потужність);
2. статусні регістри (STATUS_*);
3. події системи;
4. метрики продуктивності (counters/rates/latency).

Розподіл за топіками виконується відповідно до внутрішніх контрактів проекту; формат повідомлень узгоджується з вимогами MQTT/JSON [6], [7].

2.4. Підсистема буферизації (Store-and-Forward)

Для забезпечення відмовостійкості використовується дворівнева модель store-and-forward:

1. оперативний RAM ring buffer (швидкий рівень);
2. опційний Flash/Em_EEPROM рівень (персистентний рівень).

Алгоритм обробки в аварійному та відновлювальному режимах:

1. при недоступності MQTT нові записи зберігаються у буфер (buffer_mgr_put);
2. при заповненні RAM (і дозволеному flash-рівні) виконується spill у flash;
3. після відновлення зв'язку mqtt_gw_task виконує flush за схемою peek -> publish -> consume;
4. порядок вивантаження: спочатку flash FIFO (старіші записи), потім RAM FIFO.

Така організація дозволяє не зупиняти цикл опитування PMBus під час outage та забезпечувати контрольоване відновлення доставки після reconnect.

Фрагмент коду 2.4 - flush буфера у mqtt_gw_task.c:

```
/* Phase 1: flash first */
while (flushed < g_config.buffer.flush_batch_size &&
       flash_buffer_peek(&rec))
{
    if (!publish_json(rec.topic, rec.payload, rec.payload_len)) { break; }
    flash_buffer_consume();
    flushed++;
}

/* Phase 2: RAM second */
while (flushed < g_config.buffer.flush_batch_size &&
       buffer_mgr_peek(&rec))
{
    if (!publish_json(rec.topic, rec.payload, rec.payload_len)) { break; }
    buffer_mgr_consume();
    flushed++;
}
```

Фрагмент коду 2.5 - reconnect/backoff у mqtt_gw_task.c:

```
static void backoff_wait(void)
{
    vTaskDelay(pdMS_TO_TICKS(s_backoff_ms));
    s_backoff_ms *= 2u;
    if (s_backoff_ms > g_config.mqtt.backoff_max_ms)
```

```

    {
        s_backoff_ms = g_config.mqtt.backoff_max_ms;
    }
}

```

Таким чином, у recovery-сценарії система поєднує два механізми:

1. стабілізацію підключення через exponential backoff;
2. гарантоване FIFO-вивантаження накопичених повідомлень.

2.5. UML-моделювання архітектури ПЗ

Для графічного подання архітектури використовуються дві UML-діаграми:

1. UML component / блок-схема задач і модулів: відображає статичну структуру підсистем gateway, зв'язки між задачами та IPC-каналами.
2. UML sequence: показує динаміку обміну повідомленнями в сценарії PMBus read -> queue -> JSON -> MQTT та у сценарії outage -> buffer -> recovery -> flush.

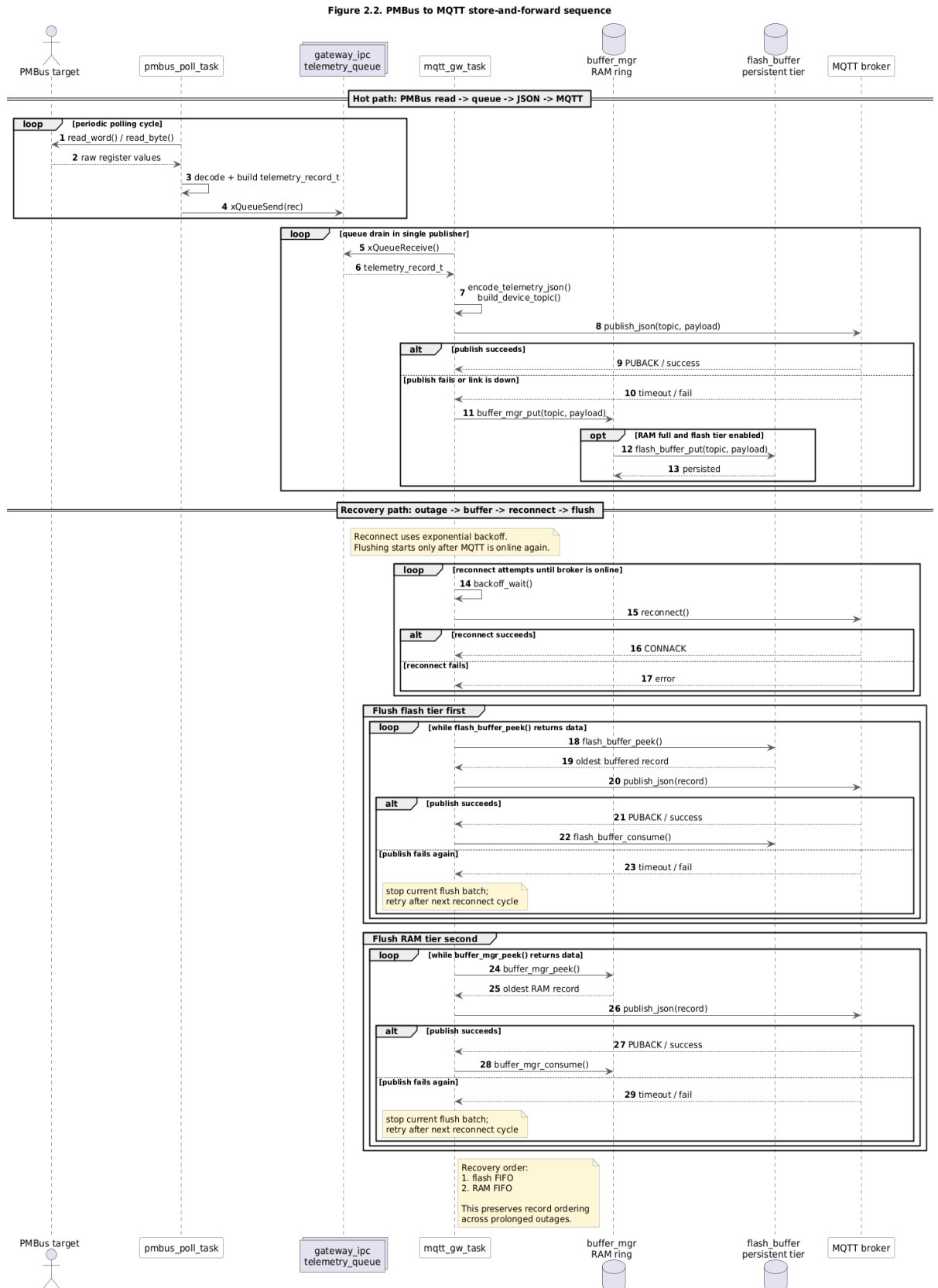


Рис. 2.2. UML Sequence Diagram для сценарію store-and-forward.

2.6. Висновки до розділу 2

У Розділі 2 сформовано архітектуру програмного забезпечення gateway на базі FreeRTOS із чітким поділом відповідальностей між задачами опитування, мережевої передачі та буферного супроводу. Показано, що така декомпозиція забезпечує:

1. детерміноване виконання критичних циклів опитування;
2. централізований контроль мережевої публікації;
3. відмовостійку доставку телеметрії через механізм store-and-forward.

Запропонована модель IPC і буферизації відповідає задачі надійного збору та доставки даних у системах моніторингу живлення. Це створює завершену основу для подальшого опису реалізації модулів у Розділі 3 та методики перевірки у Розділі 4.

РОЗДІЛ 3. РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ МІКРОКОМП'ЮТЕРА МПС

3.1. Середовище розробки та інструментальні засоби

Розроблення ПЗ мікрокомп'ютера gateway виконано на стеку Infineon ModusToolbox для платформи PSoC 6. Такий вибір забезпечує узгоджене поєднання:

1. засобів побудови прошивки (make-based build system);
2. пакетів підтримки апаратної платформи (BSP/HAL/PDL);
3. RTOS і мережевих компонентів для IoT-шлюзу.

Практичне середовище щоденної розробки у межах проєкту - Visual Studio Code з розширенням ModusToolbox Assistant. Така конфігурація дає єдину робочу точку для редагування коду, запуску команд складання, прошивання плати, перегляду журналів виконання та швидкого переходу між модулями.

З інженерної точки зору це дозволяє поєднати переваги “легкого” редактора коду з офіційним інструментарієм виробника платформи: низький час старту сесії розробки, прозорий виклик make-цілей і зручну трасованість змін між вихідними файлами, конфігурацією та результатами збірки.

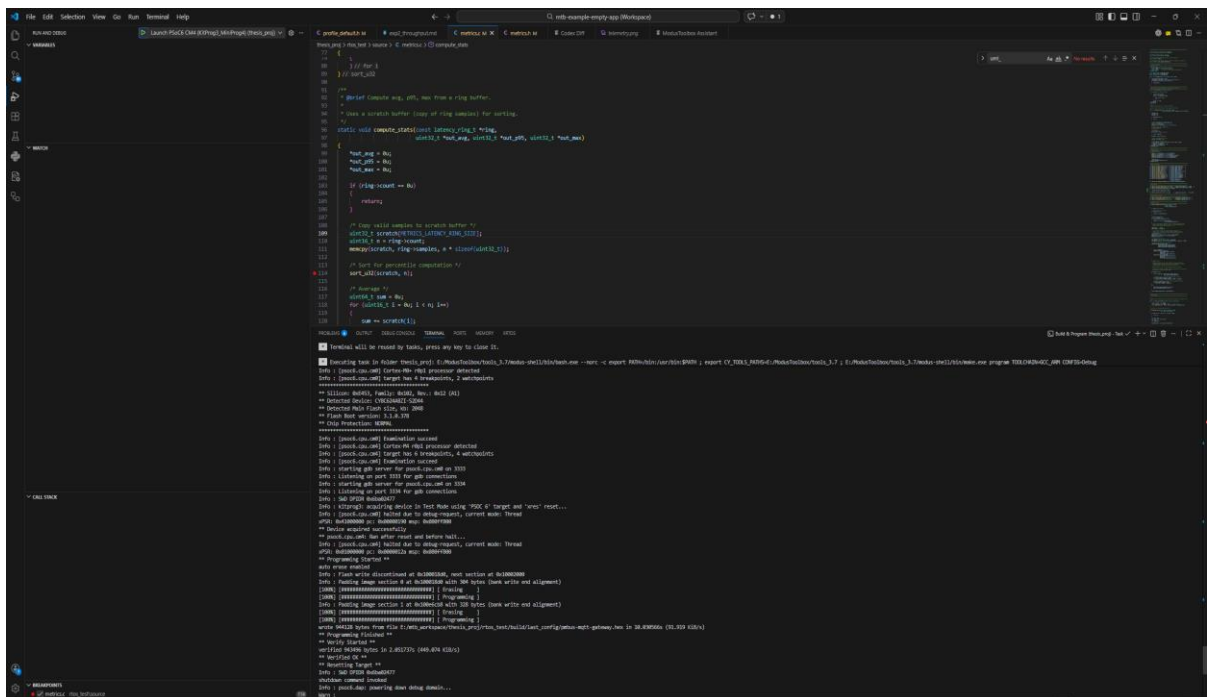


Рис. 3.1. Інтерфейс середовища розробки Visual Studio Code з розширенням

ModusToolbox Assistant під час складання, запуску та відлагодження програмних модулів gateway.

Файл: docs/coursework/figures/vscode_modustoolbox_debug.png.

У проєкті використано цільову плату CY8CKIT-062S2-43012, для якої в системі збирання явно задано відповідний BSP target. Компіляція виконується інструментальним ланцюжком GCC_ARM, що відповідає типовому workflow ModusToolbox.

Фрагмент коду 3.1 - базова конфігурація інструментарію (Makefile):

```
TARGET=APP_CY8CKIT-062S2-43012
APPNAME=pmbus-mqtt-gateway
TOOLCHAIN=GCC_ARM
CONFIG=Debug
```

```
COMPONENTS=FREERTOS LWIP MBEDTLS SECURE_SOCKETS
```

Наведені параметри визначають:

1. апаратну ціль і build-профіль;
2. ядро RTOS (FREERTOS) для багатозадачної архітектури;
3. мережевий стек LWIP і криптографічну підсистему MBEDTLS;
4. рівень secure sockets для транспортної інтеграції MQTT-клієнта.

З погляду програмної архітектури (див. Розділ 2) це середовище безпосередньо підтримує необхідні шари реалізації:

1. BSP/HAL/PDL - апаратна ініціалізація і периферія (UART, I2C, GPIO) [10];
2. FreeRTOS API - задачі, черги, синхронізація [8], [9];
3. lwIP - мережеві сервіси часу (SNTP) [14];
4. Infineon MQTT library - підключення, publish/subscribe, callback-модель [11].

Для відтворюваності експериментів у Makefile також зафіксовано профільний механізм конфігурації (GW_PROFILE), що дозволяє перемикати режими роботи gateway без зміни коду модулів:

```
ifneq ($(GW_PROFILE),)
DEFINES+= GW_PROFILE_HEADER='"profiles/profile_$(GW_PROFILE).h"'
endif
```

Це важливо для курсового проєкту, оскільки різні сценарії (штатний режим, стрес-тест, offline/recovery) можуть запускатися на однаковій кодовій базі лише через зміну профільного заголовка.

Практичні команди збирання і запуску тестів:

```
make build TOOLCHAIN=GCC_ARM CONFIG=Debug
make program
make test
```

У практичному циклі розробки ці команди застосовуються ітеративно:

1. локальне внесення змін у модуль;
2. збірка прошивки в Debug-конфігурації;
3. прошивання плати та валідація поведінки на стенді;
4. запуск host-side тестів для перевірки, що зміни не внесли регресій.

Такий цикл є важливим саме для вбудованого ПЗ, оскільки дає можливість оперативно зіставляти зміни коду з фактичною поведінкою системи на апаратурі та одночасно утримувати контроль над якістю алгоритмічної частини.

Підхід до тестування комбінований:

1. target-side збірка і запуск прошивки на стенді;
2. host-side unit-тести для критичних модулів (pmbus_decode, JSON-кодування, buffer logic), що зменшує ризик регресій ще до апаратної перевірки.

Отже, обране середовище розробки забезпечує повний інструментальний цикл: від конфігурації BSP і RTOS до мережевої взаємодії та автоматизованої перевірки частини функціональності. Це створює технологічну основу для підрозділу 3.2, де буде розглянуто послідовність ініціалізації мікрокомп'ютера від main().

3.2. Модуль ініціалізації мікрокомп'ютера

Модуль ініціалізації реалізовано у функції main() і він визначає “керований вхід” системи в робочий режим: від старту апаратної платформи до передавання керування планувальнику FreeRTOS. Для gateway це критично, оскільки

помилка на етапі boot має бути виявлена одразу (fail-fast), а не вже під навантаженням під час опитування PMBus або публікації MQTT.

У поточній реалізації послідовність запуску має такий вигляд:

1. обробка secure-специфіки (скидання WDT, якщо платформа з secure boot);
2. ініціалізація BSP і глобального переривального контексту;
3. запуск debug UART (retarget-io) і виведення boot-банера;
4. ініціалізація системних підсистем (metrics, gateway_ipc, buffer_mgr);
5. створення RTOS-задач (pmbus_poll_task, mqtt_gw_task, buffer_task, blinky_task);
6. запуск vTaskStartScheduler() з переходом у багатозадачний режим [8], [9].

Допоміжна задача blinky_task використовується як heartbeat-індикатор життєздатності системи: періодичне перемикання користувацького світлодіода підтверджує факт успішного запуску планувальника та базову працездатність RTOS-середовища.

Фрагмент коду 3.2 - апаратна ініціалізація та запуск UART-каналу (main.c):

```
/* Initialize the board support package. */
result = cybsp_init();
CY_ASSERT(CY_RSLT_SUCCESS == result);

/* Enable global interrupts. */
__enable_irq();

/* Initialize retarget-io to use the debug UART port. */
result = cy_retarget_io_init(CYBSP_DEBUG_UART_TX, CYBSP_DEBUG_UART_RX,
                           CY_RETARGET_IO_BAUDRATE);
CY_ASSERT(CY_RSLT_SUCCESS == result);
```

Цей етап формує мінімальне надійне середовище виконання: ініціалізована платформа, доступний канал журналювання і дозволені переривання. Наявність UART-логування на старті спрощує діагностику ранніх відмов, коли RTOS-задачі ще не запуснені.

Після апаратного bootstrap система переходить до ініціалізації внутрішніх підсистем із політикою “завершити запуск тільки за повної готовності”.

Фрагмент коду 3.3 - fail-fast ініціалізація IPC та буфера (main.c):

```

metrics_init();
printf("[SYS] Metrics initialised\n");

if (!gateway_ipc_init())
{
    printf("[SYS] FATAL: IPC init failed\n");
    CY_ASSERT(0);
    for (;;) { __WFI(); }
}

if (!buffer_mgr_init())
{
    printf("[SYS] FATAL: Buffer manager init failed\n");
    CY_ASSERT(0);
    for (;;) { __WFI(); }
}

```

Така логіка гарантує, що система не перейде в активну фазу збору/передачі даних без працездатного міжзадачного обміну та механізму store-and-forward. Для вбудованого шлюзу це принципово: частково ініціалізована система є небезпечнішою за керовану зупинку на етапі boot.

Після успішної підготовки інфраструктури створюються задачі і запускається планувальник.

Фрагмент коду 3.4 - створення задач і запуск планувальника (main.c):

```

xTaskCreate(pmbus_poll_task, "PMBus_Poll",
            PMBUS_POLL_TASK_STACK_SIZE, NULL,
            PMBUS_POLL_TASK_PRIORITY, NULL);

xTaskCreate(mqtt_gw_task, "MQTT_GW",
            MQTT_GW_TASK_STACK_SIZE, NULL,
            MQTT_GW_TASK_PRIORITY, NULL);

xTaskCreate(buffer_task, "Buffer",
            BUFFER_TASK_STACK_SIZE, NULL,
            BUFFER_TASK_PRIORITY, NULL);

vTaskStartScheduler();

```

Використаний порядок і пріоритезація задач відповідають архітектурному рішенню Розділу 2: опитування PMBus має вищий пріоритет за мережеву задачу, а буферний супровід працює фоновно [8], [9].

Окремо важливо зафіксувати, що частина ініціалізації виконується не в `main()`, а в тілах задач після старту планувальника:

1. `pmbus_poll_task` викликає `pmbus_init()` і піднімає I2C/PMBus контур [10];
2. `mqtt_gw_task` виконує `su_wcm_init()`, ініціалізацію MQTT-бібліотеки та встановлення сесії з брокером [11].

Це зменшує час блокування в `main()`, спрощує локалізацію помилок за підсистемами і робить старт системи більш масштабованим для подальшого розширення складу задач.

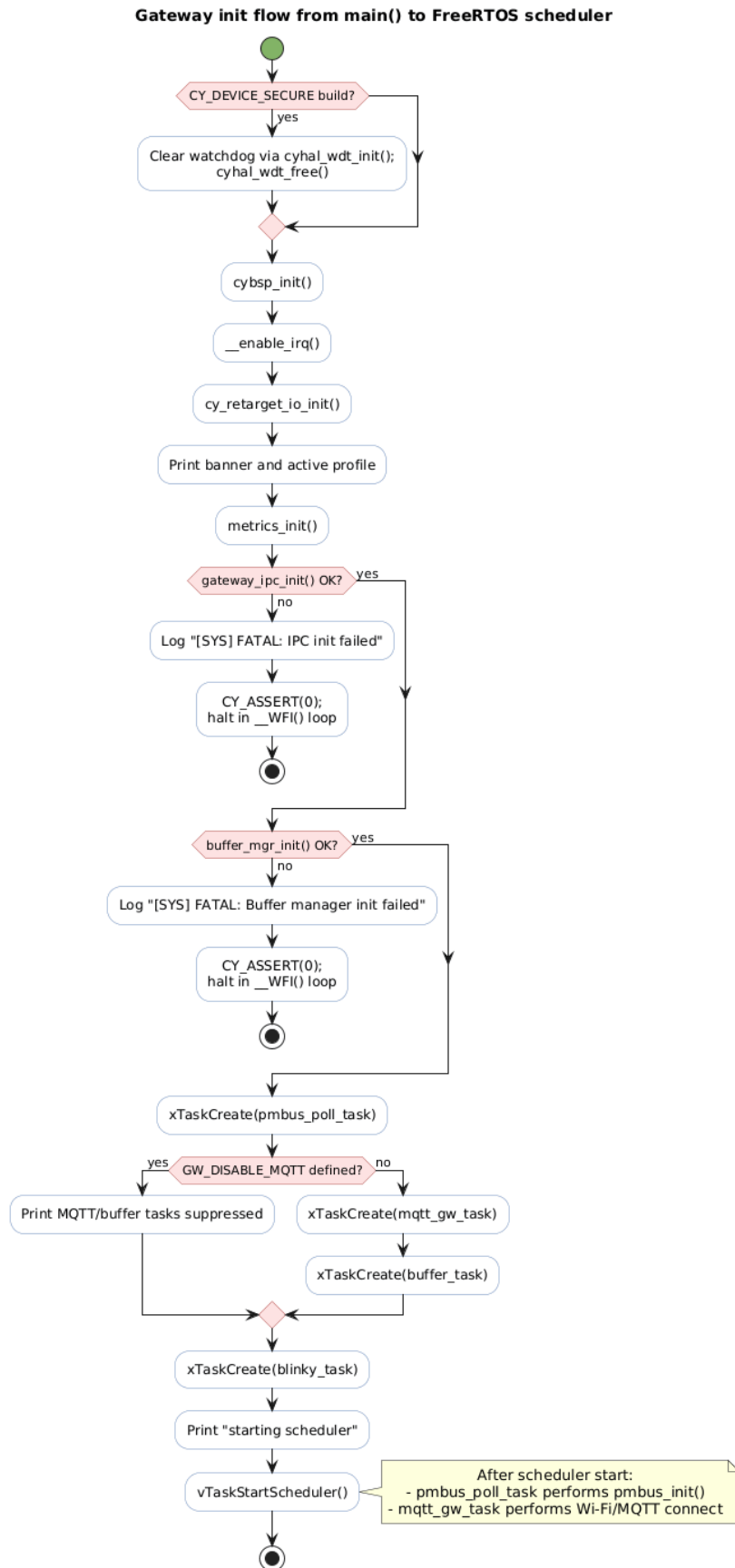


Рис. 3.2. Алгоритм ініціалізації gateway від main() до старту планувальника FreeRTOS.

3.3. Алгоритм роботи драйвера та задачі опитування PMBus

Підсистема опитування PMBus у проєкті складається з трьох взаємопов'язаних модулів:

1. `pmbus_master` - низькорівневий драйвер I2C/SMBus (ініціалізація, транзакції, `retry/timeout`, PEC, `bus-recovery`);
2. `pmbus_poll_task` - RTOS-задача періодичного опитування пристроїв і підготовки телеметричних/статусних записів;
3. `pmbus_decode` - чисті функції декодування форматів Linear11/Linear16 у фізичні величини в milli-одинацях [2], [4], [5].

Такий поділ дозволяє відокремити транспортний рівень (I2C/SMBus транзакції) від алгоритмів опитування та від математичного перетворення даних, що знижує зв'язність і підвищує тестованість коду.

Алгоритм старту PMBus-контурі починається з `pmbus_init()`, який конфігурує SCB блок мікроконтролера, налаштовує частоту шини, підключає ISR та переводить драйвер у стан “готовий до транзакцій” [10].

Фрагмент коду 3.5 - ініціалізація PMBus master (`pmbus_master.c`):

```
status = Cy_SCB_I2C_Init(PMBUS_CONTROLLER_HW, &PMBUS_CONTROLLER_config,
                        &pmbus_i2c_ctx);
if (CY_SCB_I2C_SUCCESS != status) { return PMBUS_ERR_BUS; }

uint32_t actual_rate = Cy_SCB_I2C_SetDataRate(PMBUS_CONTROLLER_HW,
                                              g_config.i2c.speed_hz,
                                              actual_scb_clk);

(void)Cy_SysInt_Init(&pmbus_scb3_irq_cfg, pmbus_scb3_isr);
NVIC_EnableIRQ(pmbus_scb3_irq_cfg.intrSrc);
Cy_SCB_I2C_Enable(PMBUS_CONTROLLER_HW);
```

У штатному профілі система опитує цільовий PMBus-пристрій за адресою 0x58, параметри шини беруться з конфігурації профілю (`speed_hz`, `timeout_ms`, `retries`, `pec_enabled`).

Базова операція драйвера - SMBus Read Word. Вона виконується у два етапи (`write cmd + repeated-start read data`) з тайм-аутом, а за увімкненого PEC - з перевіркою CRC-8 (поліном 0x07) [2], [4], [5].

Фрагмент коду 3.6 - транзакція `pmbus_read_word()` з перевіркою PEC (`pmbus_master.c`):

```
/* Phase 1: write command without STOP */
pdl_st = Cy_SCB_I2C_MasterWrite(PMBUS_CONTROLLER_HW, &wr_cfg, &pmbus_i2c_ctx);
result = wait_for_completion(pmbus_timeout_ms);

/* Phase 2: read 2 bytes (+PEC if enabled) */
pdl_st = Cy_SCB_I2C_MasterRead(PMBUS_CONTROLLER_HW, &rd_cfg, &pmbus_i2c_ctx);
result = wait_for_completion(pmbus_timeout_ms);

if (pec)
{
    uint8_t pec_input[5] = { (addr_7bit << 1) | 0u, cmd,
                             (addr_7bit << 1) | 1u, rd_buf[0], rd_buf[1] };
    if (pmbus_crc8(pec_input, 5u) != rd_buf[2]) { result = PMBUS_ERR_PEC; }
}
```

На рівні задачі `pmbus_poll_task` реалізовано детермінований цикл опитування: періодичне пробудження через `vTaskDelayUntil` з базовим кроком 10 мс, окремі дедлайни для `telemetry/status` на кожен пристрій, а також `backoff` при послідовних помилках (офлайн-поведінка) [8], [9].

Ключові особливості алгоритму:

1. кешування `VOUT_MODE` (експонента `Linear16`) з періодичним `retry`;
2. послідовне читання команд `READ_VIN`, `READ_VOUT`, `READ_IIN`, `READ_IOUT`, `READ_TEMPERATURE_1`, `READ_POUT`;
3. декодування сирих слів у `milli`-одиниці через `pmbus_linear11_to_milli()` і `pmbus_linear16_to_mv()`;
4. формування `telemetry_record_t` з полями `valid_mask`, `retries`, `read_ms`, `seq`, `ts_ms`;
5. неблокуюча відправка запису в IPC-чергу з обліком втрат при переповненні.

Фрагмент коду 3.7 - цикл читання телеметрії та публікація в IPC-чергу (`pmbus_poll_task.c`):

```
if (read_cmd(addr, PMBUS_CMD_READ_VIN, &raw, &total_retries))
{
```

```

    rec.raw_vin = raw;
    rec.vin_mV  = pmbus_linear11_to_milli(raw);
    rec.valid_mask |= TELEM_VALID_VIN;
}

if (read_cmd(addr, PMBUS_CMD_READ_VOUT, &raw, &total_retries))
{
    rec.raw_vout = raw;
    rec.vout_mV  = pmbus_linear16_to_mv(raw, state->vout_exponent);
    rec.valid_mask |= TELEM_VALID_VOUT;
}

rec.seq    = gateway_ipc_next_seq();
rec.ts_ms  = gateway_ipc_now_ms();

if (xQueueSend(gateway_ipc_telemetry_queue(), &rec, 0) != pdTRUE)
{
    metrics_inc_queue_drops();
}

```

Окремо задача виконує опитування статусних регістрів (STATUS_WORD, STATUS_VOUT, STATUS_IOUT, STATUS_TEMPERATURE) і веде облік переходів пристрою ONLINE/OFFLINE за критерієм послідовних невдалих циклів. Такий підхід зменшує “флікер” стану при короткочасних збоях на шині та полегшує аналіз експериментальних логів.

Отже, програмна реалізація 3.3 забезпечує повний ланцюг PMBus transaction -> decode -> telemetry/status record -> IPC queue, причому контур має вбудовані механізми захисту від помилок (retry, timeout, PEC, optional bus-recovery) і метрики якості обміну [2], [4], [5], [10].

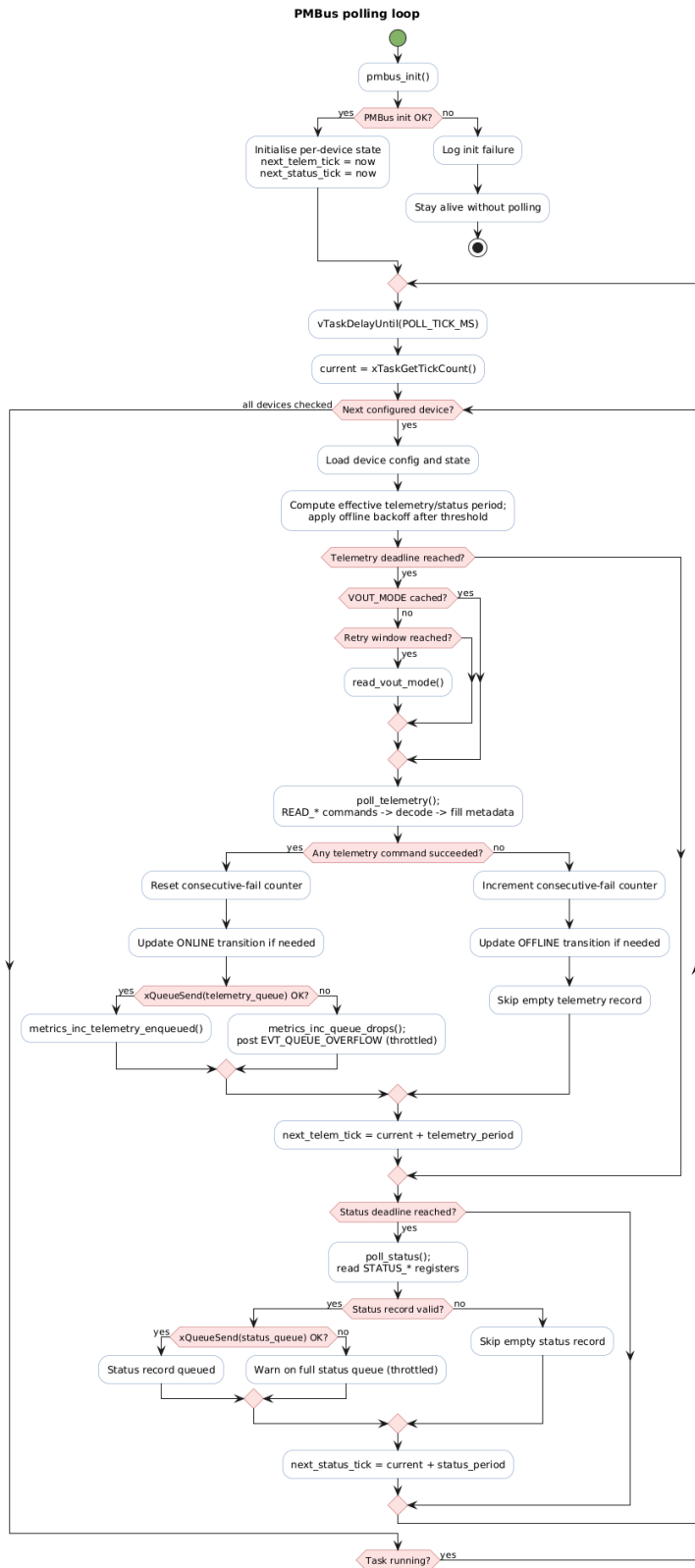


Рис. 3.3. Блок-схема алгоритму `pmbus_poll_task`: ініціалізація драйвера, опитування команд, декодування, формування записів, передача в IPC та обробка помилок.

3.4. Алгоритм мережевої взаємодії та формування JSON-контрактів

Мережеву взаємодію в gateway реалізує задача `mqtt_gw_task`, яка працює як єдиний publisher для всіх типів повідомлень (`telemetry`, `status`, `events`, `metrics`). Така серіалізація публікацій спрощує контроль QoS, облік помилок та recovery-сценарії при втраті каналу зв'язку [6], [11].

У runtime ця задача виконує чіткий життєвий цикл:

1. ініціалізація Wi-Fi підсистеми (`cy_wcm_init`);
2. підключення до точки доступу (`wifi_connect`);
3. ініціалізація MQTT-бібліотеки й створення MQTT instance;
4. з'єднання з broker;
5. робочий цикл: `drain IPC queues -> publish JSON -> flush buffer -> publish metrics`.

Фрагмент коду 3.8 - `connect/reconnect` контур у `mqtt_gw_task.c`:

```
backoff_reset();
for (;;)
{
    if (cy_wcm_is_connected_to_ap() == 0 && !wifi_connect())
    {
        backoff_wait();
        continue;
    }

    wallclock_sntp_init();

    if (!mqtt_lib_ready && !mqtt_init_and_create())
    {
        backoff_wait();
        continue;
    }

    if (!mqtt_broker_connect())
    {
        backoff_wait();
        continue;
    }
}
```

```

gateway_ipc_set_mqtt_online(true);
backoff_reset();
break;
}

```

У робочому циклі підтримується реакція на розрив сесії через callback-прапор `s_disconnect_pending`; після цього задача переходить у reconnect-шлях із exponential backoff. Це відповідає практиці побудови стійких MQTT-клієнтів у нестабільних IoT-мережах [6], [12], [13].

Фрагмент коду 3.9 - exponential backoff (mqtt_gw_task.c):

```

static void backoff_wait(void)
{
    vTaskDelay(pdMS_TO_TICKS(s_backoff_ms));
    s_backoff_ms *= 2u;
    if (s_backoff_ms > g_config.mqtt.backoff_max_ms)
    {
        s_backoff_ms = g_config.mqtt.backoff_max_ms;
    }
}

```

Модель даних у задачі публікації побудована як перетворення типізованих записів із IPC-черг у JSON-payload. Для телеметрії ланцюг виглядає так:

telemetry_record_t -> encode_telemetry_json() -> build_device_topic() -> publish_json().

Фрагмент коду 3.10 - обробка telemetry-черги (mqtt_gw_task.c):

```

int json_len = encode_telemetry_json(&rec, s_json_buf, JSON_BUF_SIZE);
int topic_len = build_device_topic(s_topic_buf, TOPIC_BUF_SIZE,
                                   rec.addr_7bit, "telemetry");

if (!publish_json(s_topic_buf, s_json_buf, (size_t)json_len))
{
    buffer_mgr_put(s_topic_buf, s_json_buf, (uint16_t)json_len);
}

```

Формат JSON для telemetry/status/events/metrics задається окремими кодерами (telemetry.c, events.c, metrics.c) і не потребує динамічних алокацій під час серіалізації (формування через `snprintf`) [7].

Фрагмент коду 3.11 - частина telemetry JSON-контракту (telemetry.c):

```

buf_printf(&pos, end,
    "{\\"ts_ms\\":%s,\\"time_synced\\":%s,\\"seq\\":%u,"
    "\\"gw_id\\":\\"%s\\",\\"addr\\":\\"0x%02X\\",\\"label\\":\\"%s\\",
    "\\"pec\\":%s,\\"read_ms\\":%u,\\"retries\\":%u",
    ts_buf, rec->time_synced ? "true" : "false",
    (unsigned)rec->seq, g_config.gw_id, (unsigned)rec->addr_7bit,
    rec->label ? rec->label : "?", rec->pec ? "true" : "false",
    (unsigned)rec->read_ms, (unsigned)rec->retries);

```

Топік-адресація будується від `base_topic` активного профілю:

1. `telemetry/status: <base_topic>/dev/0x<addr>/<suffix>`;
2. `events: <base_topic>/events`;
3. `metrics: <base_topic>/metrics`.

Фрагмент коду 3.12 - побудова device topic (telemetry.c):

```

int len = snprintf(out, out_sz, "%s/dev/0x%02X/%s",
    g_config.mqtt.base_topic,
    (unsigned)addr_7bit,
    suffix);

```

Публікація виконується через єдиний helper `publish_json_qos()`, який:

1. передає payload у `su_mqtt_publish` із заданим QoS;
2. реєструє час операції в метриках;
3. інкрементує лічильники `mqtt_pub_ok/mqtt_pub_fail`.

Для `telemetry/status/events` невдала публікація переводить запис у буфер `store-and-forward` (детальна реалізація наведена в підрозділі 3.5). Для `metrics` застосовано іншу політику: метрики не буферизуються, оскільки при затримці втрачають актуальність.

Отже, мережевий контур 3.4 реалізує стійкий шаблон `queue -> JSON -> MQTT publish -> fallback`, сумісний із моделлю `publish/subscribe MQTT` і вимогами до легковагового обміну JSON-даними у вбудованих системах [6], [7], [11].

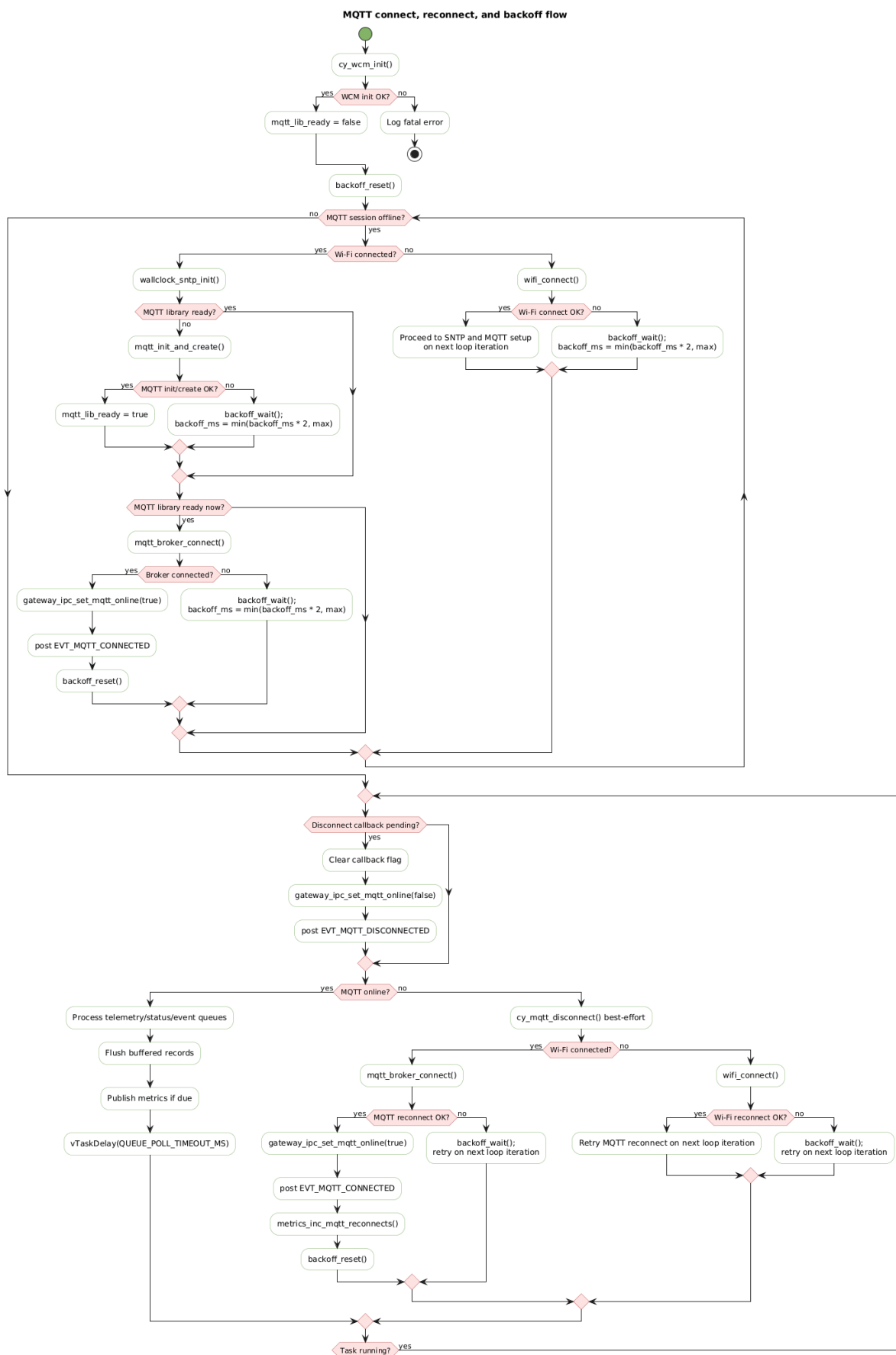


Рис. 3.4. Блок-схема алгоритму mqtt_gw_task: connect/reconnect, обробка IPC-черг, публікація JSON і fallback у буфер.

3.5. Програмна реалізація підсистеми буферизації

Підсистема буферизації реалізує шаблон store-and-forward для сценаріїв нестабільного або відсутнього мережевого підключення: дані, які не вдалося опублікувати в MQTT, не втрачаються одразу, а зберігаються локально до моменту відновлення каналу [6], [12], [13].

У проєкті використано дворівневу модель:

1. рівень 1 - RAM ring buffer (buffer_mgr.*, швидкий і volatile);
2. рівень 2 - flash persistent buffer (flash_buffer.*, повільніший, але персистентний між перезавантаженнями).

За замовчуванням (профіль default) активовано RAM-рівень, а Flash-рівень може бути ввімкнений конфігурацією flash_max_records > 0.

RAM-буфер організовано як класичне кільце з трьома індексними змінними: head (позиція запису), tail (позиція читання), count (поточна глибина). Пам'ять під масив записів виділяється один раз під час buffer_mgr_init() через pvPortMalloc, розмір визначається профілем (ram_max_records) [8], [9].

Фрагмент коду 3.13 - структура RAM-кільця і ініціалізація (buffer_mgr.c):

```
static buffer_record_t *s_ring = NULL;
static uint16_t s_capacity = 0u;
static uint16_t s_head = 0u;
static uint16_t s_tail = 0u;
static uint16_t s_count = 0u;
```

```
s_ring = (buffer_record_t *)pvPortMalloc(
    (size_t)s_capacity * sizeof(buffer_record_t));
```

Алгоритм запису buffer_mgr_put() має кілька кроків захисту від переповнення:

1. швидка перевірка доступності RAM-рівня;
2. якщо RAM повний, спроба spill у flash (flash_buffer_put) поза critical section;
3. якщо flash недоступний або теж повний - політика drop_oldest або drop_newest за конфігурацією;
4. запис topic/payload у head-слот і просування індекса.

Фрагмент коду 3.14 - overflow policy у buffer_mgr_put() (buffer_mgr.c):

```

if (s_count >= s_capacity)
{
    taskEXIT_CRITICAL();

    if (g_config.buffer.flash_max_records > 0u)
    {
        bool spilled = flash_buffer_put(topic, payload, payload_len);
        if (spilled) { return true; }
    }

    taskENTER_CRITICAL();
    if (s_count >= s_capacity)
    {
        if (g_config.buffer.drop_oldest)
        {
            s_tail = (s_tail + 1u) % s_capacity;
            s_count--;
        }
        else
        {
            taskEXIT_CRITICAL();
            return false;
        }
    }
}

```

Для збереження FIFO-семантики при вивантаженні використовується двоетапна операція peek -> consume:

1. buffer_mgr_peek() копіює найстаріший запис без видалення;
2. buffer_mgr_consume() видаляє його лише після успішної публікації.

Це виключає втрату запису між моментом читання буфера і фактичною відправкою в мережу. Синхронізація доступу до RAM-індексів реалізована короткими критичними секціями taskENTER_CRITICAL/taskEXIT_CRITICAL [8], [9].

Flash-рівень реалізовано на Em_EEPROM області (32 KB, адреса 0x14000000):

1. row 0 - метадані кільця (head, tail, count, version, crc32);

2. rows 1..63 - дані записів (по одному запису на рядок 512 B);
3. кожен data-row має magic, payload_len, topic, payload, crc32.

Під час flash_buffer_init() відбувається відновлення метаданих із валідацією magic + CRC32; у випадку невалідного стану створюється “fresh” метадані. Під час flash_buffer_peek() додатково перевіряється цілісність data-row; якщо рядок пошкоджений, tail автоматично зсувається далі, щоб уникнути зациклення на битому записі.

Фрагмент коду 3.15 - перевірка та автопропуск невалідного flash-запису (flash_buffer.c):

```
if (!record_is_valid(flash_rec))
{
    s_meta.tail = (s_meta.tail + 1u) % capacity;
    s_meta.count--;
    (void)meta_write(); /* best-effort persist */
    metrics_inc_buffer_dropped();
    return false;
}
```

Важливо, що buffer_task у поточній архітектурі не виконує sy_mqtt_publish(). Усі публікації централізовано в mqtt_gw_task, а buffer_task виконує housekeeping:

1. періодичне оновлення gauge-метрик глибини RAM/Flash;
2. підтримка фонової діагностики буфера.

Фактичне вивантаження буфера відбувається у mqtt_gw_task батчами: спочатку Flash FIFO (старіші записи), потім RAM FIFO. Це зберігає часовий порядок доставки при recovery-сценаріях.

Отже, підсистема 3.5 забезпечує контрольовану деградацію і відновлення: дані не втрачаються при коротких outage, а при тривалих збоях застосовується детермінована полісу переповнення з журналюванням подій та метрик [8], [9].

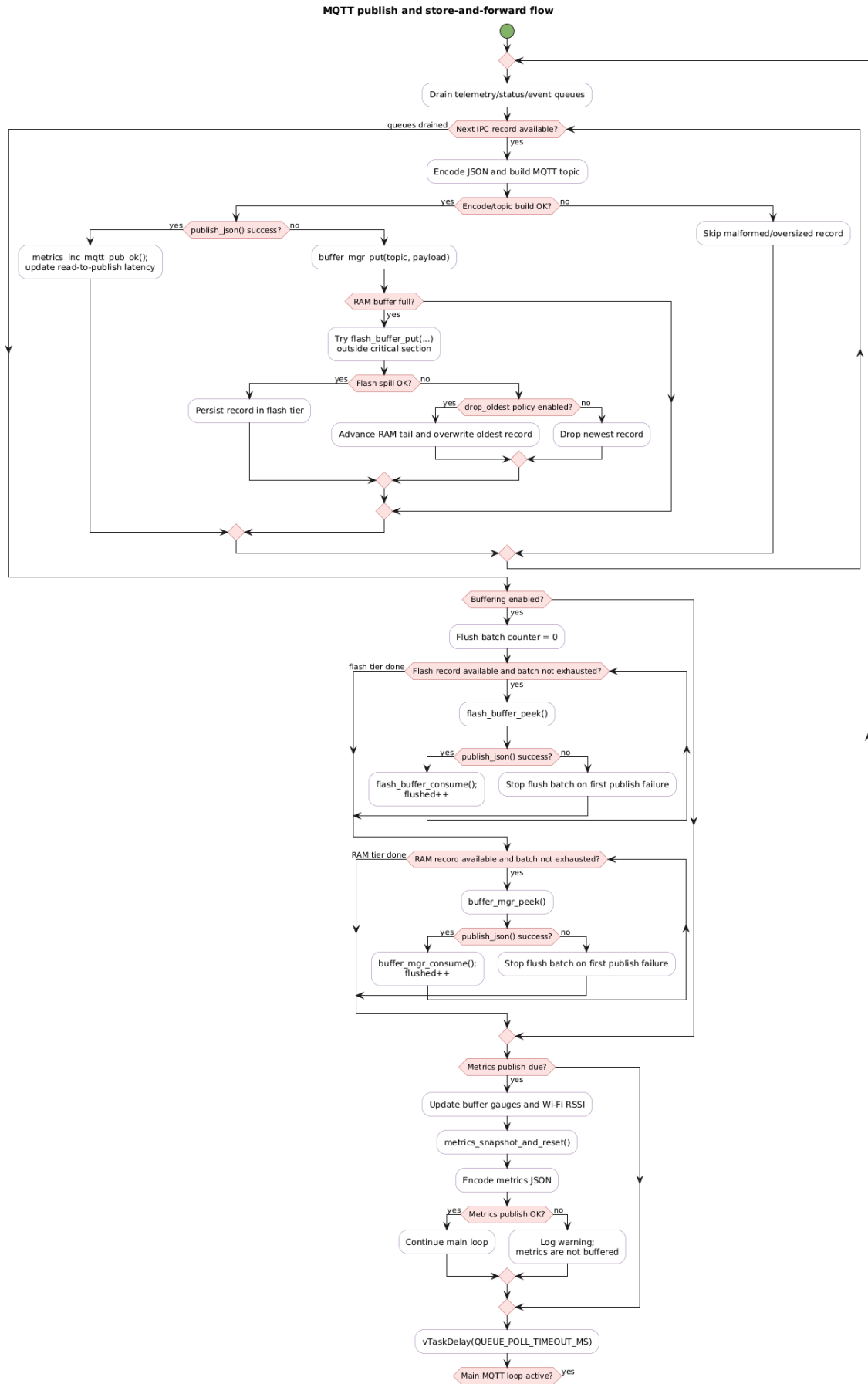


Рис. 3.5. Алгоритм store-and-forward: RAM ring buffer, spill y Flash, recovery-flush

за схемою *peek -> publish -> consume*.

3.6. Висновки до розділу 3

У Розділі 3 виконано детальне проєктно-технологічне опрацювання програмного забезпечення мікрокомп'ютера gateway: від середовища розробки і bootstrap-етапу до реалізації ключових runtime-алгоритмів збору, передавання та збереження телеметрії.

За результатами розділу встановлено:

1. модуль ініціалізації (main) забезпечує керований старт системи за fail-fast підходом із запуском FreeRTOS-задач у визначеній пріоритетній моделі;
2. PMBus-підсистема (pmbus_master + pmbus_poll_task + pmbus_decode) реалізує повний цикл read -> decode -> record -> queue з механізмами retry/timeout/PEC і контролем стану пристрою;
3. мережевий контур (mqtt_gw_task) реалізує стійку публікацію JSON-повідомлень у MQTT з reconnect/exponential-backoff та контрольованою реакцією на розриви з'єднання;
4. підсистема буферизації (RAM/Flash store-and-forward) забезпечує безперервність збору даних у offline-сценаріях і FIFO-відновлення доставки після повернення мережі.

Отже, програмна реалізація відповідає архітектурним рішенням Розділу 2 та функціональним вимогам технічного завдання, а також формує достатню практичну основу для переходу до Розділу 4, де буде наведено методику та результати тестування розробленого ПЗ.

РОЗДІЛ 4. РОЗРОБЛЕННЯ МЕТОДИКИ ТЕСТУВАННЯ ПЗ

4.1. Мета, об'єкт і завдання тестування ПЗ

Метою тестування програмного забезпечення мікрокомп'ютера gateway є підтвердження того, що розроблена система коректно реалізує функції, сформульовані в технічному завданні, і зберігає працездатність як у штатному режимі передавання телеметрії, так і в режимах деградації, пов'язаних із втратою каналу зв'язку або переповненням буфера. На відміну від неформальної перевірки “ручним” запуском, у цьому розділі фіксується відтворювана методика, яка спирається на конкретні тестові сценарії, формальні критерії приймання та об'єктивні артефакти виконання.

Об'єктом тестування є програмне забезпечення PMBus-MQTT gateway у складі модулів ініціалізації, PMBus-опитування, JSON-кодування, MQTT-публікації, міжзадачного обміну та підсистеми store-and-forward буферизації. З погляду функціональної декомпозиції перевірки підлягають як “чисті” алгоритмічні компоненти, що можуть бути протестовані на host-side без апаратного стенду, так і інтеграційна поведінка системи на рівні runtime-логів і телеметричних сесій.

У межах цього розділу тестування розглядається як засіб перевірки чотирьох ключових груп властивостей:

1. функціональної коректності обробки PMBus-даних, їх декодування та формування MQTT/JSON-повідомлень [6], [7];
2. часової та структурної коректності багатозадачної RTOS-логіки, у тому числі обміну через черги та обробки reconnect-сценаріїв [8], [9];
3. коректності взаємодії з платформними та middleware API на рівні I2C/PMBus і MQTT-клієнта [10], [11];
4. відмовостійкості підсистеми буферизації в offline/recovery режимах.

З урахуванням наявних артефактів проекту методика тестування поділяється на два взаємодоповнювальні контури. Перший контур становлять host-side unit-тести, які дозволяють ізольовано перевірити критичні алгоритми без впливу апаратної платформи. Другий контур становлять інтеграційні та

runtime-докази у вигляді лог-файлів і графіків, що відображають фактичну поведінку шлюзу в процесі опитування, буферизації та публікації даних.

Основними завданнями тестування в цьому розділі є:

1. встановити відповідність реалізованих модулів функціональним вимогам технічного завдання;
2. перевірити коректність роботи алгоритмів декодування, серіалізації та буферизації на рівні unit-тестів;
3. перевірити, що під час runtime-виконання шлюз формує валідні telemetry, status, events і metrics потоки;
4. зафіксувати формальні критерії pass/fail для кожної групи перевірок;
5. підготувати доказову базу для включення до пояснювальної записки та матеріалів захисту.

Таким чином, підрозділ 4.1 визначає логіку побудови всього Розділу 4: спочатку буде описано тестове оточення та інструменти, далі - набір тестів і сценаріїв, а після цього - результати перевірки та їх інтерпретацію.

4.2. Тестове оточення та інструментальні засоби

Тестове оточення для перевірки gateway побудовано як поєднання двох взаємодоповнювальних рівнів. Перший рівень становить host-side середовище, у якому виконуються модульні тести алгоритмічних компонентів без залежності від апаратної платформи. Другий рівень становить runtime-оточення на цільовому стенді, де перевіряється інтегрована робота задач опитування, публікації, буферизації та формування телеметричних логів.

У ролі цільового середовища використовується конфігурація, зафіксована в активному профілі profile_default.h. Відповідно до цього профілю шлюз працює з двома PMBus-пристроями за адресами 0x58 і 0x59, використовує I2C/SMBus-шину зі швидкістю 100 kHz, тайм-аут транзакції 20 ms, 2 повторні спроби, увімкнений PEC, QoS 1 для telemetry/status/events, RAM-буфер на 256 записів та період публікації metrics 10000 ms. Така конфігурація є репрезентативною для перевірки як штатного циклу опитування, так і поведінки системи при накопиченні даних у буфері [6], [8], [9], [11].

Фрагмент коду 4.1 - параметри тестового профілю (profile_default.h):

```
.i2c = {
    .speed_hz      = 100000,
    .timeout_ms    = 20,
    .retries       = 2,
    .pec_enabled   = true,
},
.mqtt = {
    .host          = "172.20.10.3",
    .port          = 1883,
    .qos_data      = 1,
},
.buffer = {
    .enabled       = true,
    .ram_max_records = 256,
    .flash_max_records = 0,
},
```

Host-side контур тестування виконується у середовищі Windows PowerShell і спирається на окрему ціль make test, яка компілює та запускає три модульні набори перевірок: test_buffer_ring, test_pmbus_decode, test_json_encode. Важливо, що ця ціль не залежить від повного стеку ModusToolbox і може виконуватися ізольовано, що підвищує відтворюваність тестування та зменшує час перевірки регресій [8], [9].

Фрагмент коду 4.2 - host-side test target (Makefile):

```
test:
$(TEST_CC) $(TEST_CFLAGS) -o test_buffer_ring \
    tests/test_buffer_ring.c && ./test_buffer_ring
$(TEST_CC) $(TEST_CFLAGS) -o test_pmbus_decode \
    tests/test_pmbus_decode.c source/pmbus_decode.c -lm \
    && ./test_pmbus_decode
$(TEST_CC) $(TEST_CFLAGS) -o test_json_encode \
    tests/test_json_encode.c source/telemetry.c source/events.c \
    source/metrics.c source/gateway_config.c source/pmbus_decode.c \
    source/gw_util.c -lm && ./test_json_encode
```

Фактичне підтвердження host-side перевірок вже зафіксовано у файлі host_test_results_2026-03-08.md, де для всіх трьох наборів тестів отримано

результат 0 failed. Це означає, що на рівні ізольованих алгоритмів перевірені декодування PMBus-значень, JSON-кодування, логіка буфера та частина метрик.

Runtime-рівень методики спирається на журнали виконання, збережені у каталозі `rtos_test/logs/`, та на побудовані за ними графічні артефакти. Для п'яти зафіксованих сесій доступні JSONL-файли потоків `telemetry`, `status`, `events`, `metrics`, а для частини сесій - рисунки `buffer.png`, `errors.png`, `latency.png`, `telemetry.png`, `throughput.png`. Саме ці артефакти використовуються як доказова база для аналізу інтеграційної поведінки системи в підрозділах 4.4 і 4.5.

Отже, тестове оточення Розділу 4 охоплює як контрольовані модульні перевірки, так і фактичні runtime-спостереження за роботою шлюзу. Це створює достатню інструментальну основу для переходу до підрозділу 4.3, де буде описано конкретний набір тестів і покриття функціональних вимог.

4.3. Методика модульного тестування host-side

Host-side модульне тестування використовується в проєкті як перший рубіж верифікації алгоритмів, які не потребують безпосереднього доступу до апаратної платформи. Такий підхід дає змогу відокремити перевірку логіки обчислень і серіалізації від перевірки периферійного та мережевого стеків, зменшуючи трудомісткість налагодження й підвищуючи відтворюваність результатів тестування [8], [9].

У межах проєкту host-side контур містить три незалежні тестові набори:

1. `test_buffer_ring` - перевірка базової логіки RAM-кільця, що лежить в основі `buffer_mgr`;
2. `test_pmbus_decode` - перевірка алгоритмів декодування PMBus/SMBus даних та обчислення PEC;
3. `test_json_encode` - перевірка контрактів JSON, `topic builder`-функцій і частини обчислення `metrics payload`.

Перший набір, `test_buffer_ring`, моделює кільцевий буфер у чистому host-side середовищі й перевіряє найкритичніші властивості алгоритму: FIFO-порядок, коректність переповнення, політики `drop_oldest` і `drop_newest`, `wrap`-

around, облік глибини та обрізання надмірно довгого payload. Саме цей набір безпосередньо покриває вимогу до відмовостійкої підсистеми store-and-forward.

Фрагмент коду 4.3 - перевірка політики drop_oldest (test_buffer_ring.c):

```
ring_put(&r, "t0", "p0", 2);
ring_put(&r, "t1", "p1", 2);
ring_put(&r, "t2", "p2", 2);

ASSERT_TRUE(ring_put(&r, "t3", "p3", 2),
            "put on full w/ drop_oldest returns true");

ring_get(&r, &out);
ASSERT_EQ_STR(out.payload, "p1",
            "oldest (p0) was dropped, p1 is first");
```

Другий набір, test_pmbus_decode, перевіряє математичну коректність функцій pmbus_linear11_to_milli, pmbus_linear16_to_mv, pmbus_linear11_from_milli, pmbus_vout_mode_exponent і pmbus_crc8. Методика цього набору базується на контрольних значеннях, round-trip перевірках і допустимих похибках квантування, що природно виникають для PMBus-форматів Linear11 і Linear16 [2], [4], [5].

У практичному сенсі саме цей набір доводить, що gateway коректно переводить сирі слова PMBus у фізичні величини, які далі публікуються в MQTT.

Третій набір, test_json_encode, перевіряє сформовані контракти обміну: наявність обов'язкових полів у telemetry/status/event/metrics payload, відсутність полів для невалідних вимірів, коректність форматування чисел, topic builder-функції та поведінку при замалому вихідному буфері. Таким чином перевіряється відповідність внутрішніх структур даних вимогам MQTT/JSON-моделі обміну [6], [7].

Фрагмент коду 4.4 - перевірка часткового telemetry payload (test_json_encode.c):

```
rec.valid_mask = TELEM_VALID_VOUT;
int len = encode_telemetry_json(&rec, buf, sizeof(buf));

TEST_ASSERT_TRUE(json_contains(buf, "\"vout\":3.30"));
TEST_ASSERT_TRUE(!json_contains(buf, "\"vin\""));
```

```
TEST_ASSERT_TRUE(!json_contains(buf, "\"iin\""));
TEST_ASSERT_TRUE(!json_contains(buf, "\"temp1\""));
```

Для систематизації покриття вимог host-side тести можна зіставити з функціональними модулями таким чином.

Компонент / вимога	Тестовий набір	Що саме перевіряється
Буферизація та FIFO-поведінка	test_buffer_ring	FIFO, overflow, wrap-around, truncation, depth
Декодування PMBus і PEC	test_pmbus_decode	Linear11/Linear16, VOUT_MODE, CRC-8/PEC, похибки round-trip
MQTT/JSON контракти	test_json_encode	telemetry/status/event/metrics JSON, topic builders, error on small buffer

Критерієм проходження host-side модульного тестування є одночасне виконання трьох умов:

1. усі тестові набори успішно компілюються в host-side середовищі;
2. кожен тестовий виконуваний файл завершується без аварійного завершення;
3. підсумковий результат кожного набору містить 0 failed.

Водночас потрібно зафіксувати межі цієї методики: host-side тести не перевіряють безпосередньо роботу периферії I2C, Wi-Fi стеку, MQTT middleware і часову поведінку системи на реальному стенді. Саме тому після модульного рівня в наступному підрозділі розглядається інтеграційне та runtime-тестування.

4.4. Методика інтеграційного та runtime-тестування

Після завершення host-side перевірок наступним етапом є інтеграційне та runtime-тестування на цільовому стенді. На відміну від модульного рівня, де оцінюються окремі алгоритми в ізоляції, тут перевіряється узгоджена робота всіх основних підсистем gateway: задачі pmbus_poll_task, задачі mqtt_gw_task, черг gateway_ipc, механізму буферизації та зовнішньої MQTT-інфраструктури. Фактично цей рівень показує, чи переходить коректність окремих модулів у

коректну поведінку цілісної RTOS-системи під робочим навантаженням [6], [8], [9], [10], [11].

Методика runtime-тестування в межах проєкту базується не на ручному спостереженні за консольним виводом, а на аналізі артефактів сесії, збережених у каталозі `rtos_test/logs/`. Кожна сесія містить JSONL-журнали потоків `telemetry`, `status`, `events`, `metrics`, а для частини сесій також похідні графічні артефакти `buffer.png`, `errors.png`, `latency.png`, `telemetry.png`, `throughput.png`. Такий підхід є принципово важливим, оскільки він дозволяє відтворювано перевіряти поведінку системи після завершення експерименту, а також використовувати збережені журнали як доказову базу в пояснювальній записці.

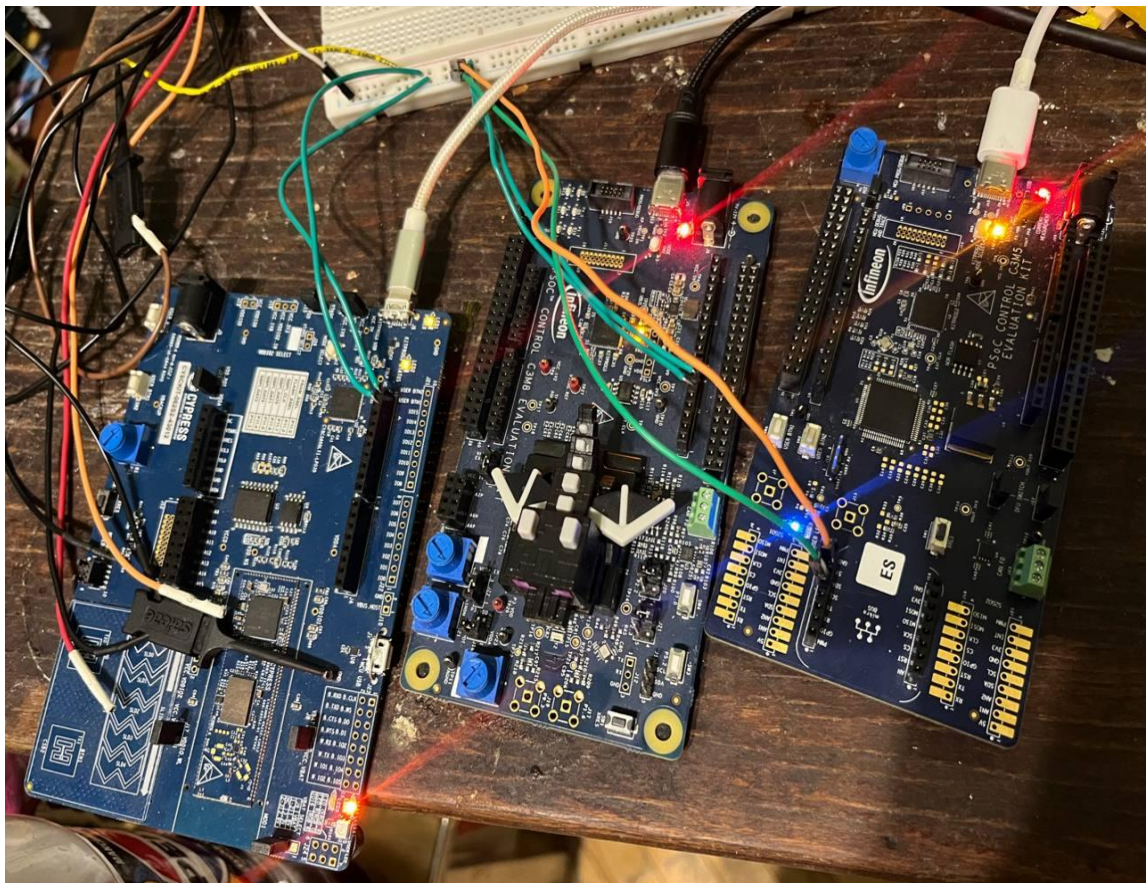


Рис. 4.1. Зібраний стенд для runtime-тестування. Шлюз(зліва) та 2 цільові пристрої

Базовий сценарій інтеграційної перевірки виконується у такій послідовності:

1. на цільову плату завантажується прошивка gateway з активним профілем `profile_default.h`;

2. забезпечується доступність PMBus target-пристроїв та MQTT broker;
3. шлюз запускається у штатному online-режимі на інтервалі, достатньому для накопичення декількох вікон метрик і репрезентативної вибірки telemetry;
4. журнали telemetry.jsonl, status.jsonl, events.jsonl, metrics.jsonl зберігаються для подальшого аналізу;
5. на основі журналів будуються графіки та виконується формальна перевірка критеріїв коректності.

У межах цієї методики оцінюються чотири типи runtime-потоків.

1. telemetry: перевіряється наявність коректного topic, адреси пристрою, часової мітки, номера послідовності, ознаки рес, часу читання read_ms, кількості повторних спроб retries та набору фізичних параметрів. Для telemetry потоку також контролюється монотонність seq і відсутність структурних пошкоджень JSON.
2. status: перевіряється формування окремого потоку статусних повідомлень із полями status_word, status_vout, status_iout, status_temperature. Для штатного режиму допустимим вважається нульовий статус, тоді як ненульові прапорці мають трактуватися як ознака аварійного або попереджувального стану підсистеми живлення [2], [4], [5].
3. events: використовується як канал фіксації подій шлюзу. У штатних безаварійних сесіях допустима відсутність записів у events.jsonl; у деградованих сценаріях саме цей потік має відображати reconnect, buffer overflow, transport failure або інші аномалії.
4. metrics: є основним інструментом інтеграційної діагностики. За цим потоком перевіряються лічильники pmbus_reads_ok/fail, mqtt_pub_ok/fail, mqtt_reconnects, buffer_enqueued/dequeued/dropped, queue_drops, глибина буфера, часові характеристики read_to_publish_*, pmbus_txn_*, mqtt_publish_* та інтенсивності повідомлень.

Фрагмент журналу 4.5 - приклад telemetry- та status-повідомлень runtime-сесії (logs/20260228_190001/*.jsonl):


```

{"ts_ms":676037,"seq":406,"gw_id":"gw01","addr":"0x58","label":"psu_a","pec":true,
"read_ms":12,"retries":0,"v":{"vin":47.5,"vout":11.89},"a":{"iin":2.03,"iout":8.25},"
c":{"temp1":40.5},"w":{"pout":98.12},"_topic":"pmbus/gw01/dev/0x58/telemetry"}
{"ts_ms":680045,"seq":409,"gw_id":"gw01","addr":"0x58","label":"psu_a","status_word":"0
x0000","status_vout":"0x00","status_iout":"0x00","status_temperature":"0x00","_topic":"
pmbus/gw01/dev/0x58/status"}

```

Наведений фрагмент показує, що інтеграційна методика перевіряє не лише факт наявності MQTT-повідомлень, а й структурну повноту контрактів даних: коректну адресацію пристрою, часову прив'язку, сервісні поля обміну та фізичні значення, отримані з PMBus-пристрою.

Фрагмент журналу 4.6 - приклад metrics-повідомлення для вікна спостереження близько 10 с (logs/20260228_190001/metrics.jsonl):

```

{"window_ms":10024,"counters_delta":{"pmbus_reads_ok":34,"pmbus_reads_fail":0,"mq
tt_pub_ok":7,"mqtt_pub_fail":0,"mqtt_reconnects":0,"buffer_enqueued":0,"buffer_dequeued
":0,"buffer_dropped":0,"queue_drops":0},"gauges":{"buffer_depth_ram":0,"buffer_depth_fl
ash":0,"wifi_rssi_dbm":-50},"timing_ms":{"read_to_publish_avg":90.1,"read_to_publish_p9
5":109.0,"pmbus_txn_avg":12.0,"mqtt_publish_avg":62.2},"rates":{"telemetry_msgs_per_s":
0.4,"pmbus_cmds_per_s":3.3}}

```

Саме потік metrics дає змогу формалізувати runtime-перевірку. Якщо на рівні telemetry і status підтверджується правильність структури повідомлень, то на рівні metrics перевіряється системна якість виконання: відсутність помилок читання, відсутність втрат черг, стабільність часу доставки та адекватна глибина буфера. Для сценаріїв деградації й відновлення аналізуються ті самі поля, але з іншою інтерпретацією: збільшення mqtt_pub_fail, mqtt_reconnects, buffer_enqueued та buffer_dequeued повинно корелювати з графіками buffer.png і errors.png, а очищення буфера після відновлення з'єднання повинно підтверджуватися поверненням buffer_depth_* до нульових або штатних значень.

Окреме місце у методиці займає аналіз графічних артефактів. Графіки telemetry.png та throughput.png використовуються для візуального підтвердження стабільності потоку даних, latency.png - для оцінки затримок від читання PMBus до MQTT-публікації, errors.png - для фіксації збоїв і повторних спроб, а buffer.png - для контролю динаміки локального накопичення даних. У тексті записки

доцільно використовувати 2-3 найбільш репрезентативні графіки, а повний комплект винести до додатків.

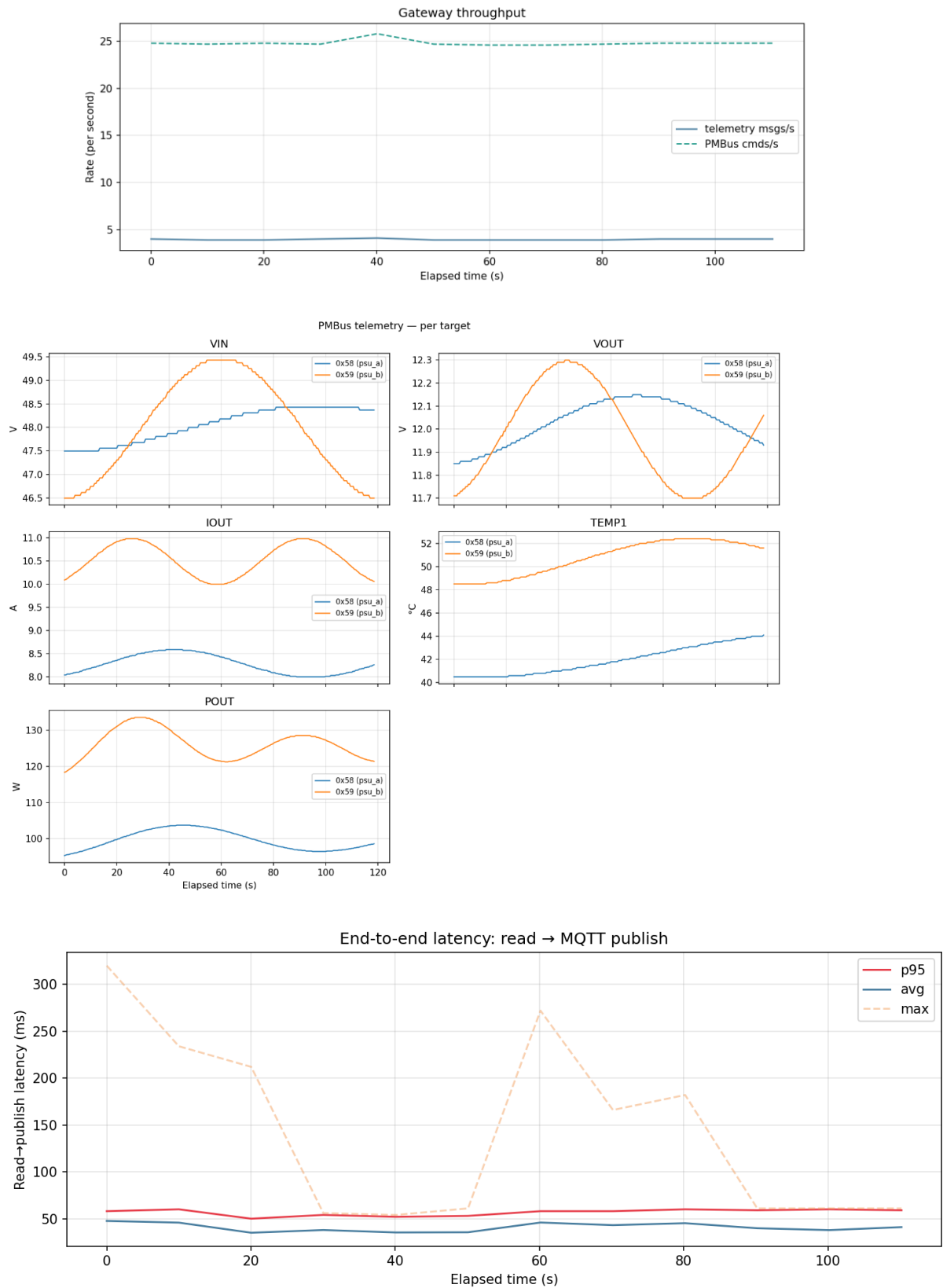


Рис. 4.2. Приклад runtime-артефактів тестової сесії: графіки telemetry, latency, throughput, errors та buffer.

Таким чином, підрозділ 4.4 задає формалізовану процедуру інтеграційного тестування: від запуску стендової сесії до аналізу JSONL-журналів і графіків. На цій основі в підрозділі 4.5 можуть бути вже наведені конкретні результати перевірки, їх числові значення та висновки щодо відповідності ПЗ вимогам технічного завдання.

4.5. Результати тестування, критерії pass/fail та інтерпретація

У цьому підрозділі узагальнюються фактичні результати перевірки gateway на двох рівнях: host-side модульному та інтеграційному runtime-рівні. Такий поділ принциповий, оскільки дозволяє окремо оцінити коректність алгоритмів і окремо - якість їх спільної роботи в умовах реального виконання на стенді.

Підсумкові результати host-side наборів наведено в табл. 4.1.

Таблиця 4.1. Підсумкові результати host-side модульних тестів

Тестовий набір	Кількість успішних перевірок	Кількість помилок	Висновок
test_buffer_ring.exe	71	0	Пройдено
test_pmbus_decode.exe	48	0	Пройдено
test_json_encode.exe	66	0	Пройдено
Разом	185	0	Пройдено

Отриманий результат 185 passed, 0 failed означає, що на рівні ізольованих алгоритмів підтверджено коректність трьох критичних підсистем:

1. кільцевого буфера та політик переповнення;
2. декодування PMBus-форматів і розрахунку PEC;
3. JSON-контрактів telemetry/status/events/metrics.

Тобто всі формальні критерії модульного рівня виконано без зауважень. З погляду трасування вимог це означає, що програмна реалізація буферизації,

PMBus-декодування та MQTT/JSON серіалізації має підтверджену алгоритмічну коректність ще до виходу на стендові випробування.

Для інтеграційного рівня ключовими доказами є збережені runtime-сесії. За даними інвентаризації логів, у каталозі `rtos_test/logs/` зафіксовано п'ять сесій, з яких три містять повний комплект графічних артефактів. Узагальнення доступних runtime-доказів наведено в табл. 4.2.

Таблиця 4.2. Наявні runtime-сесії та їх придатність для аналізу

Сесія	telemetry	status	events	metrics	Figures	Висновок
20260228_163025	15	3	0	3	0	Коротка online-сесія
20260228_163133	10	2	0	2	5	Коротка online-сесія з графіками
20260228_163534	149	30	0	30	5	Репрезентативна online-сесія
20260228_190001	150	30	0	30	5	Репрезентативна online-сесія
20260228_193947	15	3	0	3	0	Коротка online-сесія

Інтерпретація цих даних є такою. По-перше, runtime-докази достатні для підтвердження стабільної роботи gateway у штатному online-режимі: наявні сесії містять послідовні потоки telemetry, status та metrics, а також графіки, побудовані на їх основі. По-друге, всі зафіксовані events.jsonl порожні, що узгоджується з безаварійним ходом експериментів, але водночас означає відсутність у поточному пакеті доказів окремої runtime-сесії з деградацією каналу зв'язку та відновленням доставки.

Для кількісної оцінки штатного режиму було проаналізовано репрезентативні сесії 20260228_163534 і 20260228_190001. Їхні metrics-повідомлення демонструють однакову картину:

1. `pmbus_reads_ok` = 34 за вікно близько 10 с, при цьому `pmbus_reads_fail` = 0, `pmbus_timeouts` = 0, `pmbus_nack` = 0, `pmbus_crc_pec_fail` = 0;
2. `mqtt_pub_ok` = 7 при `mqtt_pub_fail` = 0 і `mqtt_reconnects` = 0;
3. `buffer_enqueued` = 0, `buffer_dequeued` = 0, `buffer_dropped` = 0, `queue_drops` = 0;
4. `buffer_depth_ram` = 0, `buffer_depth_flash` = 0, що підтверджує відсутність потреби в буферизації у штатному online-сценарії;
5. середня затримка `read_to_publish_avg` перебуває приблизно в межах 90.0-101.5 ms, а `read_to_publish_p95` - в межах 108-122 ms;
6. середній час PMBus-транзакції `pmbus_txn_avg` становить близько 12 ms, а середній час MQTT-публікації `mqtt_publish_avg` - близько 55-67 ms;
7. рівень сигналу Wi-Fi `wifi_rssi_dbm` перебуває в діапазоні від -53 до -47 dBm, що для лабораторного стенду є достатнім для стабільного передавання повідомлень.

Окремо перевірено структурну коректність `telemetry` і `status` потоків. У збережених `telemetry`-повідомленнях присутні обов'язкові поля `gw_id`, `addr`, `label`, `seq`, `pec`, `read_ms`, а також вкладені групи фізичних параметрів `v`, `a`, `c`, `w`. Значення `status_word`, `status_vout`, `status_iout`, `status_temperature` у зафіксованих `status`-повідомленнях дорівнюють нулю, що відповідає штатному режиму роботи PMBus target без активних попереджень чи аварій [2], [4], [5].

На основі наведених результатів формальні критерії `pass/fail` можуть бути зафіксовані так.

1. Критерій `host-side unit tests` = 0 failed: виконано, статус - PASS.
2. Критерій відповідності JSON `topic/payload` контрактам: виконано на `host-side` та підтверджено `runtime`-журналами, статус - PASS.
3. Критерій коректності FIFO та `overflow`-поведінки буфера: виконано на модульному рівні, статус - PASS.

4. Критерій стабільного online-циклу PMBus -> JSON -> MQTT: виконано, статус - PASS.
5. Критерій runtime-підтвердження сценарію offline -> buffer -> recovery -> flush: на рівні поточного пакета артефактів повністю не підтверджено, статус - PARTIALLY VERIFIED.

Останній пункт потребує окремого пояснення. Архітектурно і алгоритмічно store-and-forward механізм уже перевірений модульними тестами та описаний у Розділах 2 і 3, однак збережені runtime-сесії не містять ненульових mqtt_reconnects, buffer_enqueued, buffer_dequeued або buffer_dropped. Отже, для фінального пакета захисту доцільно додатково підготувати окрему контрольовану outage/recovery-сесію з відключенням broker або мережі, щоб підтвердити цей сценарій інтеграційно, а не лише алгоритмічно.

Таким чином, результати тестування вже дозволяють зробити обґрунтований висновок про коректність основної функціональності gateway у штатному режимі та про високу якість його модульної реалізації. Водночас пакет доказів для сценарію відновлення після втрати зв'язку слід посилити окремим експериментом, що є типовою та технічно коректною практикою для систем класу store-and-forward.

4.6. Висновки до розділу 4

У Розділі 4 розроблено формалізовану методику тестування програмного забезпечення PMBus-MQTT gateway, яка охоплює як модульний, так і інтеграційний рівні перевірки. На відміну від неформального підходу, коли працездатність системи оцінюється лише за фактом запуску, запропонована методика спирається на відтворювані сценарії, формальні критерії pass/fail та об'єктивні артефакти у вигляді журналів і графіків.

За результатами host-side тестування підтверджено коректність критичних алгоритмів буферизації, декодування PMBus-даних і формування JSON-контрактів: усі три тестові набори завершилися з результатом 0 failed. Це означає, що ключові алгоритмічні компоненти системи працюють коректно та не містять виявлених логічних дефектів на рівні ізольованої перевірки.

Інтеграційне runtime-тестування на основі сесій telemetry, status, metrics і похідних графіків показало, що у штатному online-режимі шлюз стабільно виконує цикл PMBus -> decode -> JSON -> MQTT, не демонструючи помилок читання, втрат у чергах, збоїв публікації або неконтрольованого зростання буфера. Зафіксовані числові показники затримок, пропускну здатності та якості Wi-Fi-з'єднання підтверджують придатність реалізованого ПЗ до задач телеметричного моніторингу в реальному часі.

Водночас результати розділу показали і межу поточного пакета доказів: сценарій offline -> buffer -> recovery -> flush у наявних runtime-сесіях не був зафіксований окремим контрольованим експериментом. Тому для фінального пакета захисту доцільно додатково підготувати окрему outage/recovery-сесію, яка підтвердить відмовостійкість підсистеми store-and-forward не лише на модульному, а й на інтеграційному рівні.

Отже, програмне забезпечення gateway у його поточному стані успішно пройшло модульну та штатну інтеграційну перевірку, а розроблена методика тестування є достатньою основою для обґрунтування працездатності системи в пояснювальній записці та матеріалах до захисту.

ВИСНОВКИ

У курсовій роботі розроблено програмне забезпечення мікрокомп'ютера PMBus-MQTT gateway, призначене для збору телеметрії з цифрової підсистеми керування живленням і передавання цих даних до MQTT-інфраструктури через Wi-Fi. Поставлену мету досягнуто: сформовано функціональну структуру системи, обрано апаратну платформу CY8CKIT-062S2-43012, визначено інтерфейсну модель I2C/SMBus + PMBus + Wi-Fi + MQTT та обґрунтовано режими роботи gateway у станах ініціалізації, штатної роботи, локальної буферизації та відновлення після збою каналу зв'язку.

У межах роботи розроблено програмну архітектуру на базі FreeRTOS з чітким поділом відповідальностей між задачами `pmbus_poll_task`, `mqtt_gw_task` і `buffer_task`. Реалізовано модулі ініціалізації системи, драйвер PMBus master з підтримкою `retry`, `timeout` і PEC, алгоритми циклічного опитування пристроїв, формування `telemetry/status/event/metrics` повідомлень, а також механізм `store-and-forward` буферизації для підвищення відмовостійкості телеметричного каналу.

Практичним результатом роботи є працездатна *firmware*-реалізація gateway, яка забезпечує перетворення сирих PMBus-даних у структуровані MQTT/JSON-повідомлення та придатна для використання у лабораторних і дослідницьких стендах моніторингу джерел живлення. Запропонована побудова є масштабованою: вона може бути адаптована як до одного PMBus target, так і до конфігурацій із кількома пристроями та іншими профілями опитування без зміни базових архітектурних принципів.

Розроблена методика тестування підтвердила коректність ключових підсистем. Усі *host-side* модульні тести завершилися з результатом 0 failed, що підтвердило правильність алгоритмів буферизації, PMBus-декодування та JSON-кодування. Інтеграційне *runtime*-тестування показало стабільну роботу gateway у штатному *online*-режимі без помилок читання PMBus, втрат у чергах та збоїв MQTT-публікації. Це дає підстави вважати, що розроблене програмне

забезпечення відповідає основним вимогам технічного завдання та є придатним до подальшого використання в межах поставленої задачі.

Окремо зафіксовані в технічному завданні три цільові числові параметри також досягнуто. По-перше, у штатному профілі підтверджено одночасне опитування 2 PMBus target-пристроїв за адресами 0x58 і 0x59, тобто вимогу не менше 2 виконано. По-друге, за репрезентативними runtime-сесіями отримано `read_to_publish_p95` у межах 108-122 ms, що задовольняє цільовий поріг не більше 150 ms. По-третє, у контрольному online-сеансі лічильники `pmbus_reads_fail`, `queue_drops` та `mqtt_pub_fail` дорівнювали нулю, а отже сума критичних помилок обміну також становила 0, як і вимагалось технічним завданням.

Разом з тим у роботі зафіксовано і практично важливе обмеження: сценарій `offline -> buffer -> recovery -> flush` у поточному пакеті runtime-логів не підтверджено окремою контрольованою `outage/recovery`-сесією. Тому подальшим напрямом розвитку є проведення такого експерименту на стенді, а також розширення графічної та доказової частини записки. Загалом результати курсової роботи свідчать про успішне розв'язання поставленої інженерної задачі та створюють завершену основу для захисту проєкту.

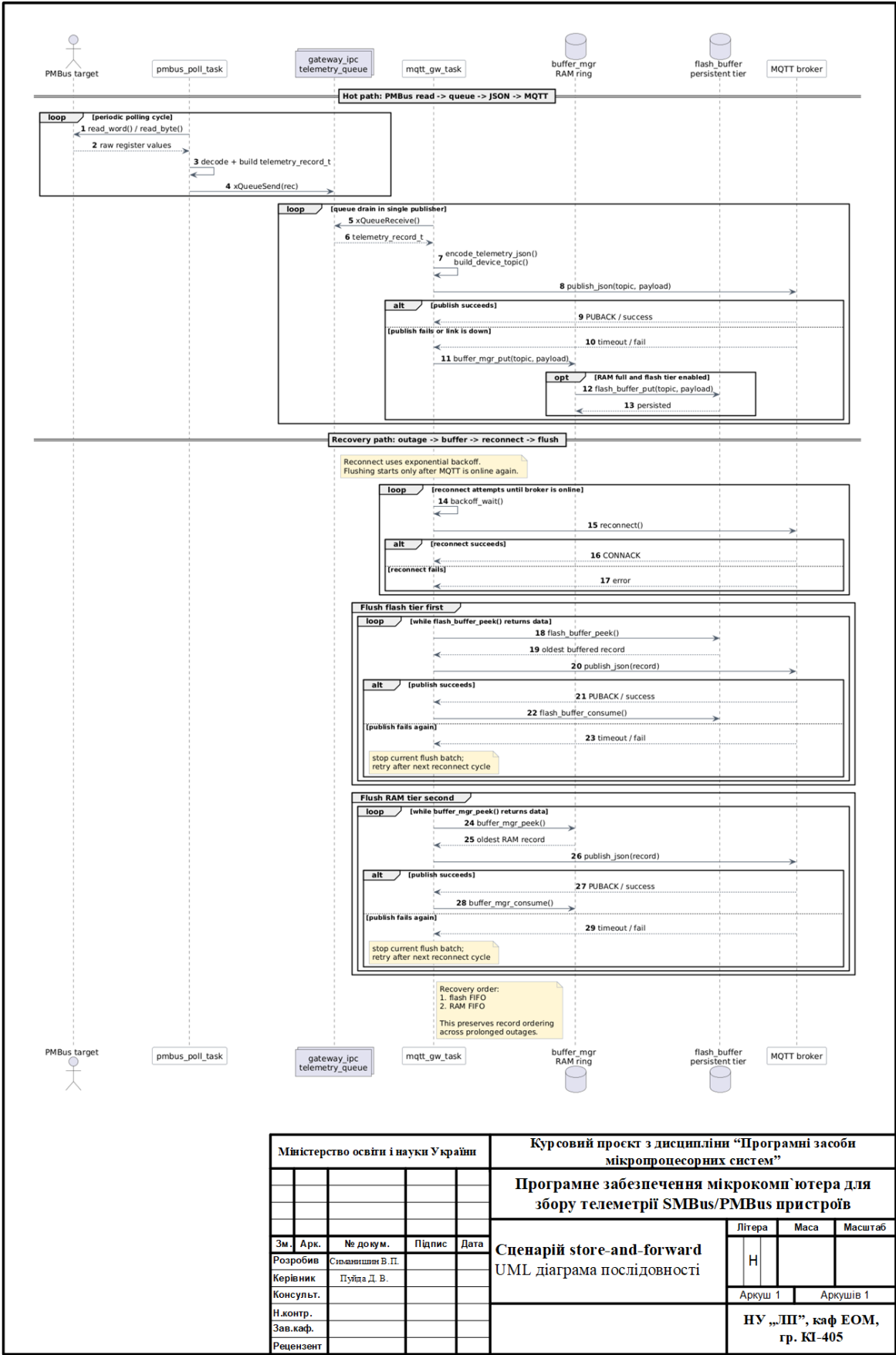
СПИСОК ЛІТЕРАТУРИ

1. Методичні вказівки до курсового проєкту з дисципліни «Програмні засоби мікропроцесорних систем». Львів: НУ «Львівська політехніка», 2025.
2. Power Management Bus Implementers Forum. PMBus Specification Part II: Command Language, Rev. 1.3.1, 18-Jun-2015. URL: <https://pmbusprod.wpenginepowered.com/wp-content/uploads/2022/01/PMBus-Specification-Rev-1-3-1-Part-II-20150313.pdf>
3. Power Management Bus Implementers Forum. PMBus Current Specifications. URL: <https://pmbus.org/specification-archives/>
4. SMBus.org. System Management Bus (SMBus) Specification, Version 3.3.1, October 20, 2024. URL: https://smbus.org/specs/SMBus_3_3_1_20241020.pdf
5. NXP Semiconductors. UM10204 I2C-bus specification and user manual, Rev. 7.0, 1 October 2021. URL: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>
6. OASIS. MQTT Version 5.0. OASIS Standard, 07 March 2019. URL: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.pdf>
7. Bray T. The JavaScript Object Notation (JSON) Data Interchange Format. RFC 8259, IETF, December 2017. URL: <https://www.rfc-editor.org/rfc/rfc8259>
8. FreeRTOS. FreeRTOS Documentation (Kernel API). URL: <https://docs.freertos.org/>
9. FreeRTOS. xTaskCreate API Reference. URL: <https://www.freertos.org/Documentation/02-Kernel/04-API-references/01-Task-creation/01-xTaskCreate>
10. Infineon Technologies. PSoC 6 Peripheral Driver Library: SCB I2C API Reference. URL: https://infineon.github.io/psoc6pdl/pdl_api_reference_manual/html/group__group__scb__i2c.html

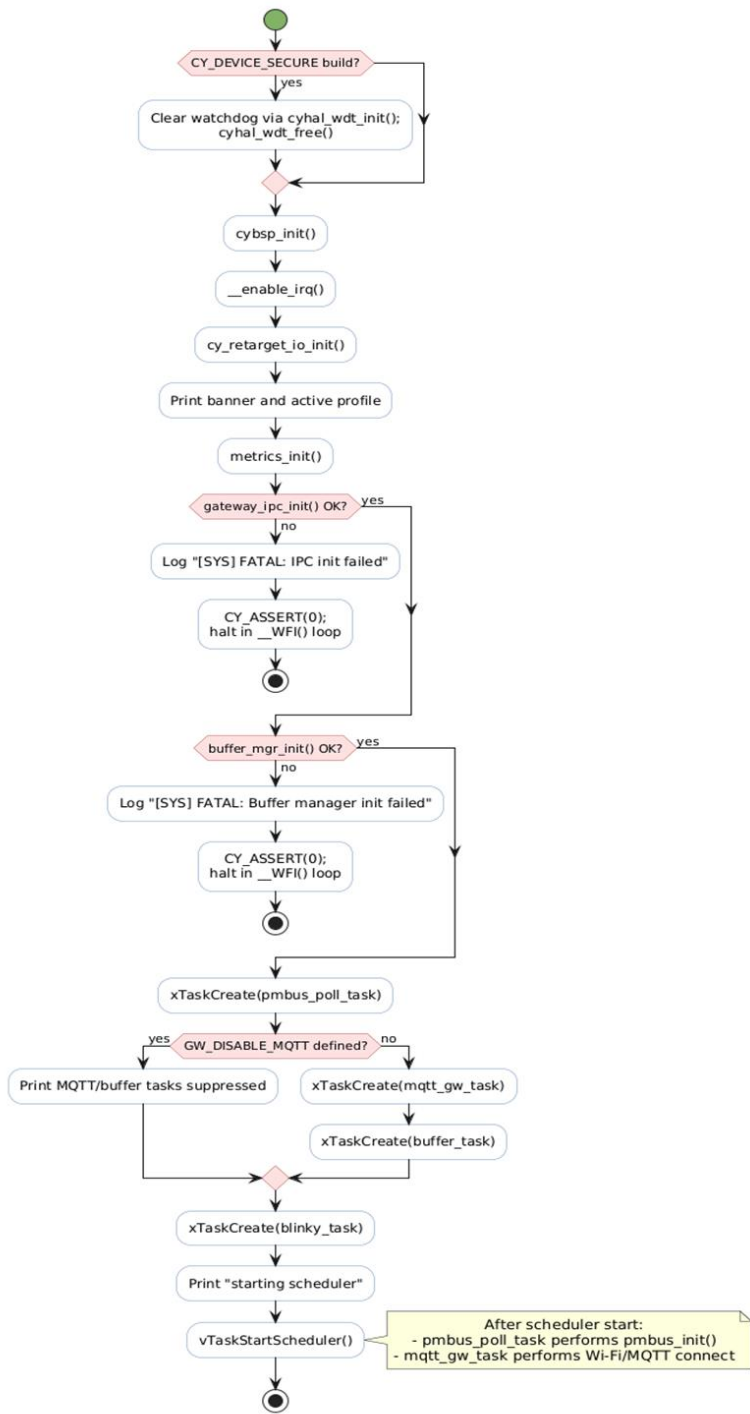
11. Infineon Technologies. MQTT Client Library API Reference (MTB shared mqtt). URL:
https://infineon.github.io/mqtt/api_reference_manual/html/index.html
12. Thangavel D., Ma X., Valera A., Tan H.-X., Tan C. K.-Y. Performance evaluation of MQTT and CoAP via a common middleware. 2014 IEEE Ninth International Conference on Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP), 2014. DOI: 10.1109/ISSNIP.2014.6827678. URL: <https://doi.org/10.1109/ISSNIP.2014.6827678>
13. Silva I., Leandro R., Macedo D., Guedes L. A Performance Analysis of Internet of Things Networking Protocols: Evaluating MQTT, CoAP, OPC UA. Applied Sciences. 2021;11(11):4879. DOI: 10.3390/app11114879. URL: <https://doi.org/10.3390/app11114879>
14. lwIP Project. SNTP API (group__sntp). URL:
https://www.nongnu.org/lwip/2_1_x/group__sntp.html
15. Infineon Technologies. CY8CKIT-062S2-43012 - Evaluation Board (PSOC 62S2 Wi-Fi BT Pioneer Kit). URL: <https://www.infineon.com/evaluation-board/CY8CKIT-062S2-43012>

ДОДАТКИ

Додаток А – UML Діаграма для сценарію store-and-forward

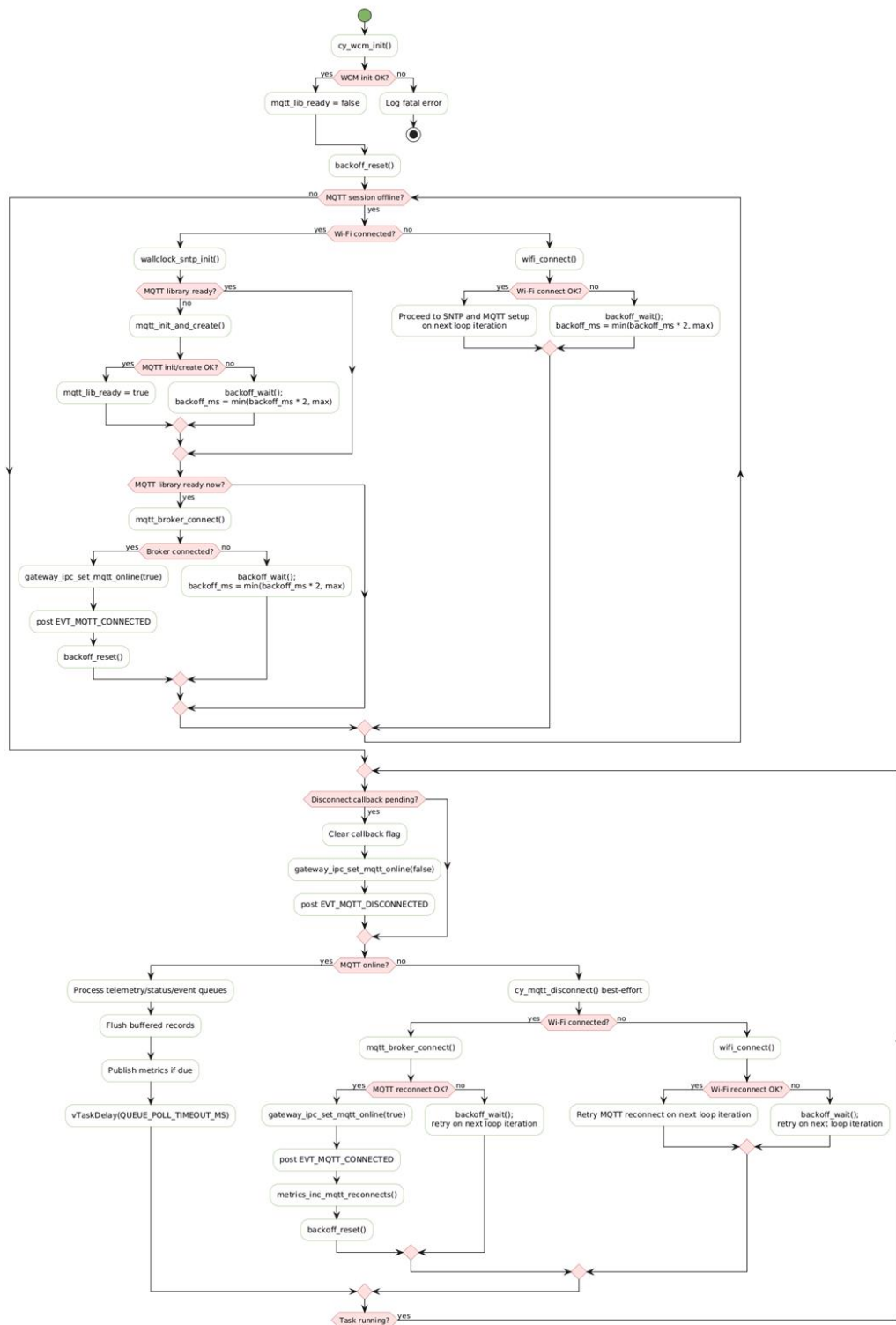


Додаток Б – Схема алгоритму ініціалізації шлюзу



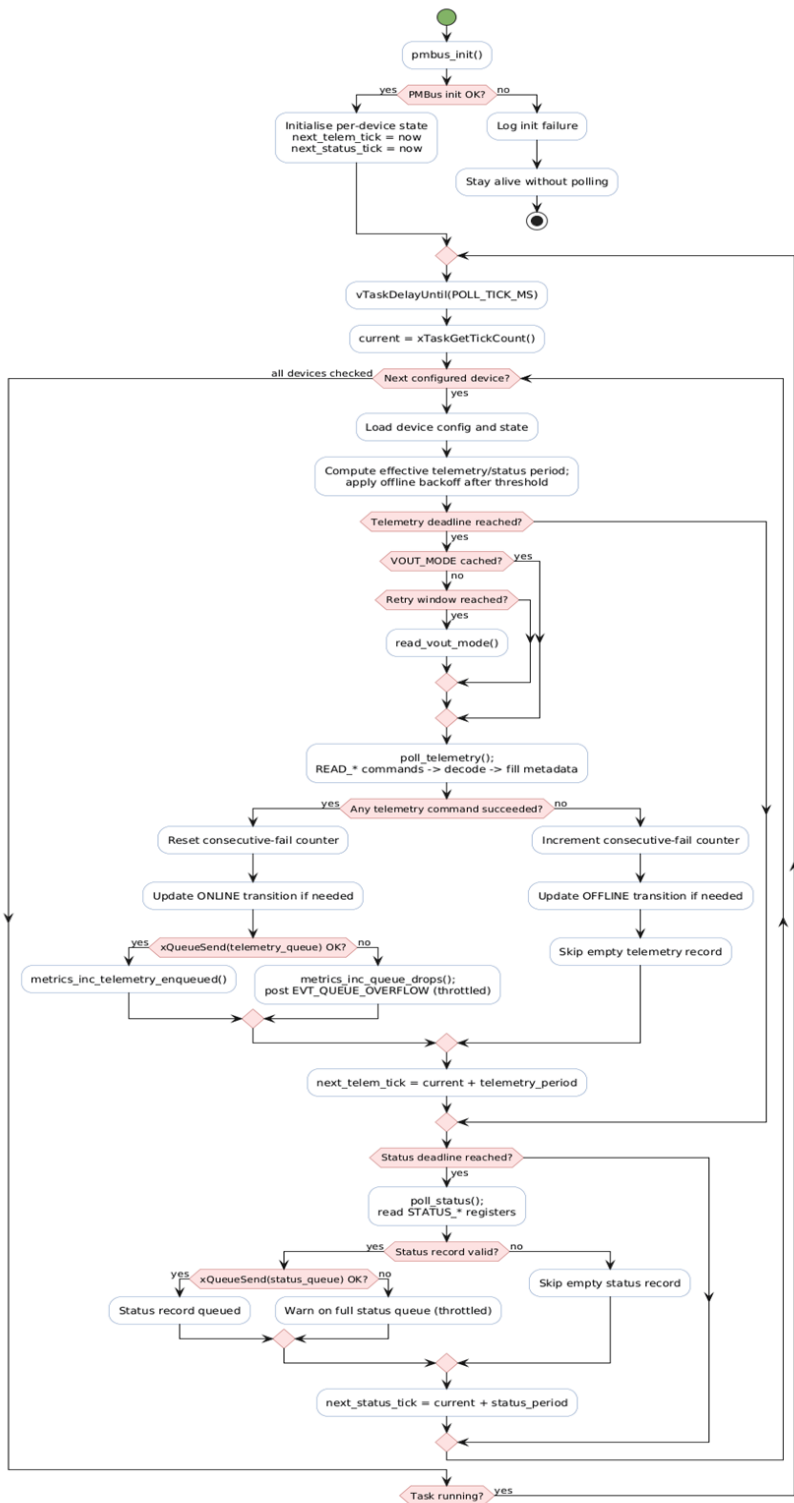
Міністерство освіти і науки України					Курсовий проєкт з дисципліни “Програмні засоби мікропроцесорних систем”			
					Програмне забезпечення мікрокомп'ютера для збору телеметрії SMBus/PMBus пристроїв			
					Ініціалізації gateway від main() до старту планувальника FreeRTOS <i>Схема алгоритму</i>	Літера	Маса	Масштаб
Зм.	Арк.	№ докум.	Підпис	Дата		H		
Розробив	Сиданишин В.П.							
Керівник	Пуйда Д.В.							
Консульт.						Аркуш 1	Аркушів 1	
Н.контр.					НУ „ІП”, каф ЕОМ, гр. КІ-405			
Зав.каф.								
Рецензент								

Додаток В – Схема алгоритму роботи з MQTT



Міністерство освіти і науки України					Курсовий проєкт з дисципліни “Програмні засоби мікропроцесорних систем”		
					Програмне забезпечення мікрокомп'ютера для збору телеметрії SMBus/PMBus пристроїв		
					mqtt_gw_task: connect, обробка IPC-черг, публікація JSON і fallback у буфер Схема алгоритму	Літера	Маса
						Н	
						Аркуш 1	Аркушів 1
Зм.	Арк.	№ докум.	Підпис	Дата	НУ „ЛПІ”, каф ЕОМ, гр. КІ-405		
Розробив		Славинський В.П.					
Керівник		Пуйпа Д.В.					
Консульт.							
Н.контр.							
Зав.каф.							
Рецензент							

Додаток Г – Схема алгоритму опитування пристроїв PMBus



Міністерство освіти і науки України					Курсовий проєкт з дисципліни “Програмні засоби мікропроцесорних систем”			
					Програмне забезпечення мікрокомп'ютера для збору телеметрії SMBus/PMBus пристроїв			
					rmbus_poll_task: ініціалізація та робота драйвера Схема алгоритму			
Зм.	Арк.	№ докум.	Підпис	Дата				
Розробив		Сьоманішин В.П.						
Керівник		Пуйда Д.В.						
Консульт.								
Н.контр.					НУ „ЛП”, каф ЕОМ, гр. КІ-405			
Зав.каф.								
Рецензент								

Додаток Д – Лістинг програмного модуля

Репозиторій проєкту на GitHub:

<https://github.com/reRocketR/edge-gateway-pmbus>

Додаток Г – Схема електрична функціональна

